

From Database to Analysis: Unveiling Insights in Examination Systems

A Comprehensive Journey from Data Collection to Strategic Decision-Making



Marwa Ashraf - Omar Genedy - Ghada H. Eldaoudy
- Adel Ramadan - Mohamed Ghanem

TABLE OF CONTENTS:

1. Database Design

- 1.1 Overview
- 1.2 Requirements
- 1.3 The ER-Diagram
- 1.4 Mapping and Normalization

2. Physical Implementation of Database using (SSMS)

- 2.1 Creation of Tables and Constraints
- 2.2 Populating Sample Data

3. T-SQL Data Manipulation and Views Management

- 3.1 Data Manipulation Language (DML) Operations
- 3.2 Stored Procedures for DML Operations
- 3.3 Views Creation

4. Reporting

- 4.1 Creating Reports Using stored procedures.
- 4.2 Visual Reporting with Visual Studio

5. Data Warehouse

- 5.1 Design Overview: multidimensional data modelling
- 5.2 Implementing the Data Warehouse
- 4.3 ETL Processes: Extraction, Transformation, and Loading

6. Power BI Integration

- 6.1 Data Modelling for Analytics
- 6.2 Visualization Techniques for Data Insights

7. Python Application Development

- 7.1 Establishing Database Connections with Python
- 7.2 Building Frontend and Backend Applications

1. DATABASE DESIGN

1.1 "Overview"

An examination system for ITI student instructors encompasses various components to ensure effective tracking, programming, and departmental management.

Here's an overview:

- 1.Student Database Management:** The system should maintain a comprehensive database of student instructors, including personal information, training history, and current enrolment status.
- 2.Exam Scheduling and Programming:** It should facilitate the scheduling of exams for student instructors, considering their training curriculum and progress. The system should allow administrators to set exam dates, times, ensuring proper planning and organization.
- 3. Course Material Management:** The system should provide a platform for managing course materials, including syllabi, study guides, and reference materials. This ensures that student instructors have access to relevant resources to prepare for their exams effectively.
- 4.Tracking of Student Progress:** The system should track the progress of student instructors throughout their training program. Performance in coursework, and completion of practical exercises or projects.
- 5. Examination Platform:** Implementing an examination platform allows students to take exams. The system should support various question formats, including MCQ, True/False
- 6. Grading and Evaluation:** The system should automate the grading process and provide instant feedback to students upon completing their exams. It should also generate detailed performance reports for instructors to review.
- 7. Departmental Collaboration:** The system should facilitate collaboration among different departments involved in the training of students. This includes sharing exam schedules, coordinating resources, and communicating important updates or changes.
- 8. Scalability and Flexibility:** As the number of student instructors and training programs may vary over time, the system should be scalable to accommodate growth. Additionally, it should be flexible enough to adapt to changes in curriculum, regulations, or technological advancements.

By incorporating these features into the examination system for ITI student instructors, institutions can effectively manage exams, track student progress, and ensure the quality of training programs offered.

1.2 “REQUIREMENTS”

Student

- Each student is uniquely identified by St_ID.
- Attributes include SSN, First Name, Last Name, Graduation Year, Gender, Date of Birth, St_Email, Password, and Phone.

Job

- If a student has a job, it is appended in the table.
- Jobs are characterized by Job_ID, Job Title, Salary, Hiring Date, Company Name, and Job Type.

Freelance

- Each student should have at least one freelance job.
- Freelance jobs are characterized by Fr_ID, Description, Cost, Duration, St_Account, Client Name, and Client Number.

Certificate

- Each student should have at least one certificate.
- Certificates are characterized by Credential URL, Fr_ID, Ct_Name, Description, Hours, and Ct_ID.

Branch

- Each branch is uniquely identified by B_ID.
- It has a name, denoted as B_Name.

Address

- Each branch is uniquely identified by Address_ID.
- It has a City Name.

Department

- Each department is uniquely identified by Dep_ID.
- It has a name, represented by Dep_Name, and is managed by a Manager.

Program

- Departments offer one or more programs.
- Programs are characterized by P_ID and P_Name.
- A course is exclusively offered by many programs.

Track

- Tracks are characterized by T_ID, T_Name, Duration, Capacity, and No_Courses.
- Each track includes a list of courses.
- A track is exclusively offered by one program.

Course

- Courses have a unique identifier Topic_ID and a Cr_Name.
- Each course is associated with many tracks.

Topic

- Courses have a unique identifier Cr_ID and a Cr_Name.
- Each course is associated with many tracks.

Company

- Each branch has a deal with at least one company.
- It has Company_ID, Company_Name, Launch_Date, and Company_Field.

✚ **Evaluation Projects**

- Courses have evaluation projects.
- Evaluation projects have characteristics Proj_ID, Pro_Name, Min_Mark, and Full_Mark.

✚ **Graduation Project**

- Every track should have a graduation project.
- Attributes include Project_ID, Project_Name, MinMarktoPass, and Fullmark.

✚ **Graduation Team**

- Each student should be in a team.
- Attributes include Team_ID, Score, Grade, P_SDate, and P_EDate.

✚ **Exam**

- Each course has an exam.
- Exam characteristics are Exam_ID, Duration, No. Questions, Date, Min_Mark, and Full_Mark.

✚ **Questions**

- Each course has many questions.
- Question characteristics are Q_ID, Question_Type, Choices 1-4, and Correct_Choice.

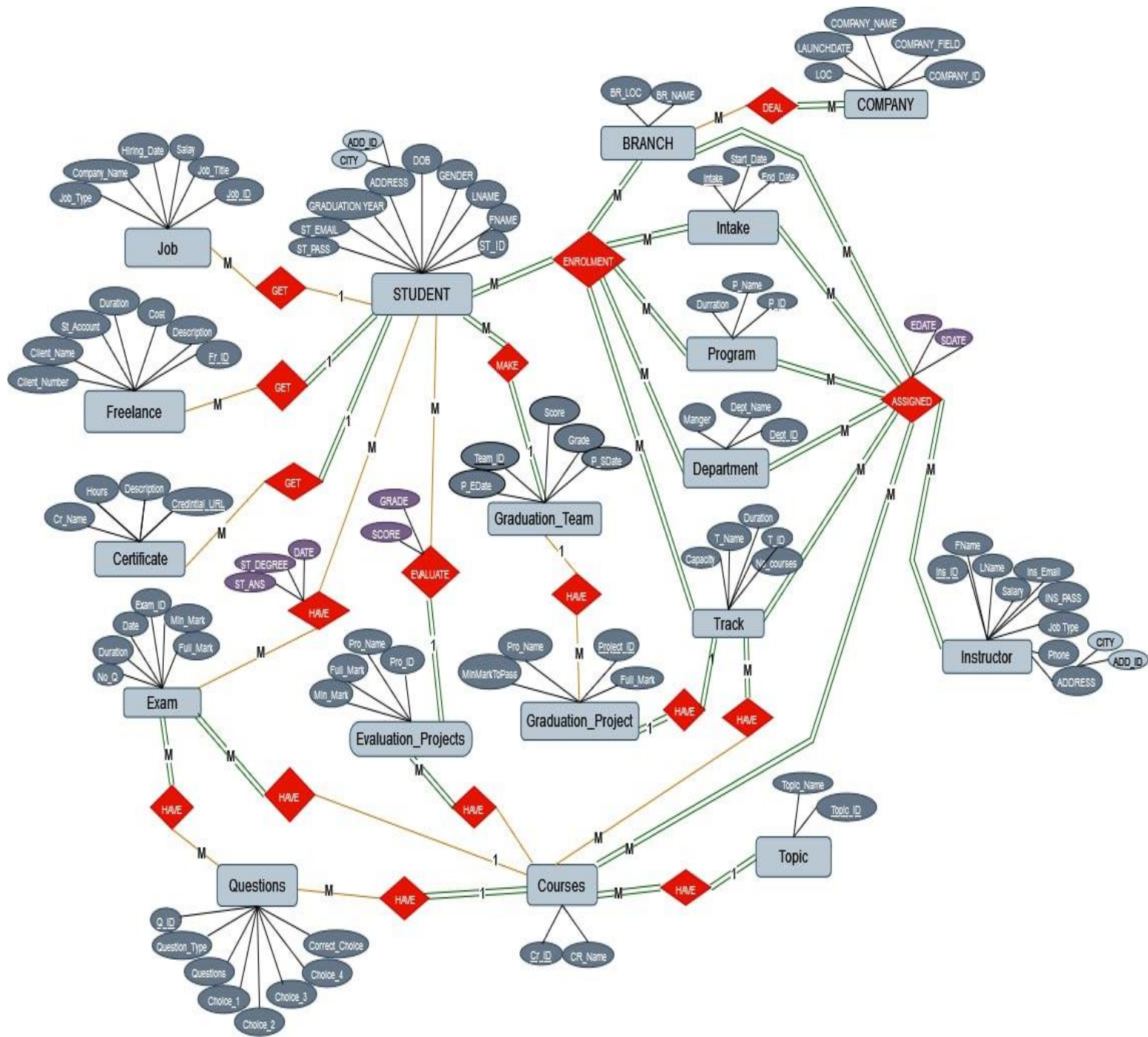
✚ **Intake**

- Enrollment in a track should be accompanied by enrollment in an intake.
- Intake characteristics include Intake, Start_Date, and End_Date.

✚ **Instructor**

- Instructors belong to one branch and can teach many courses.
- They have a unique identifier Ins_ID and attributes such as Ins_SSN, First_Name, Last_Name, Salary, Ins_Email, Password, Phone, and Job Type

1.3 “THE ER_DIAGRAM”

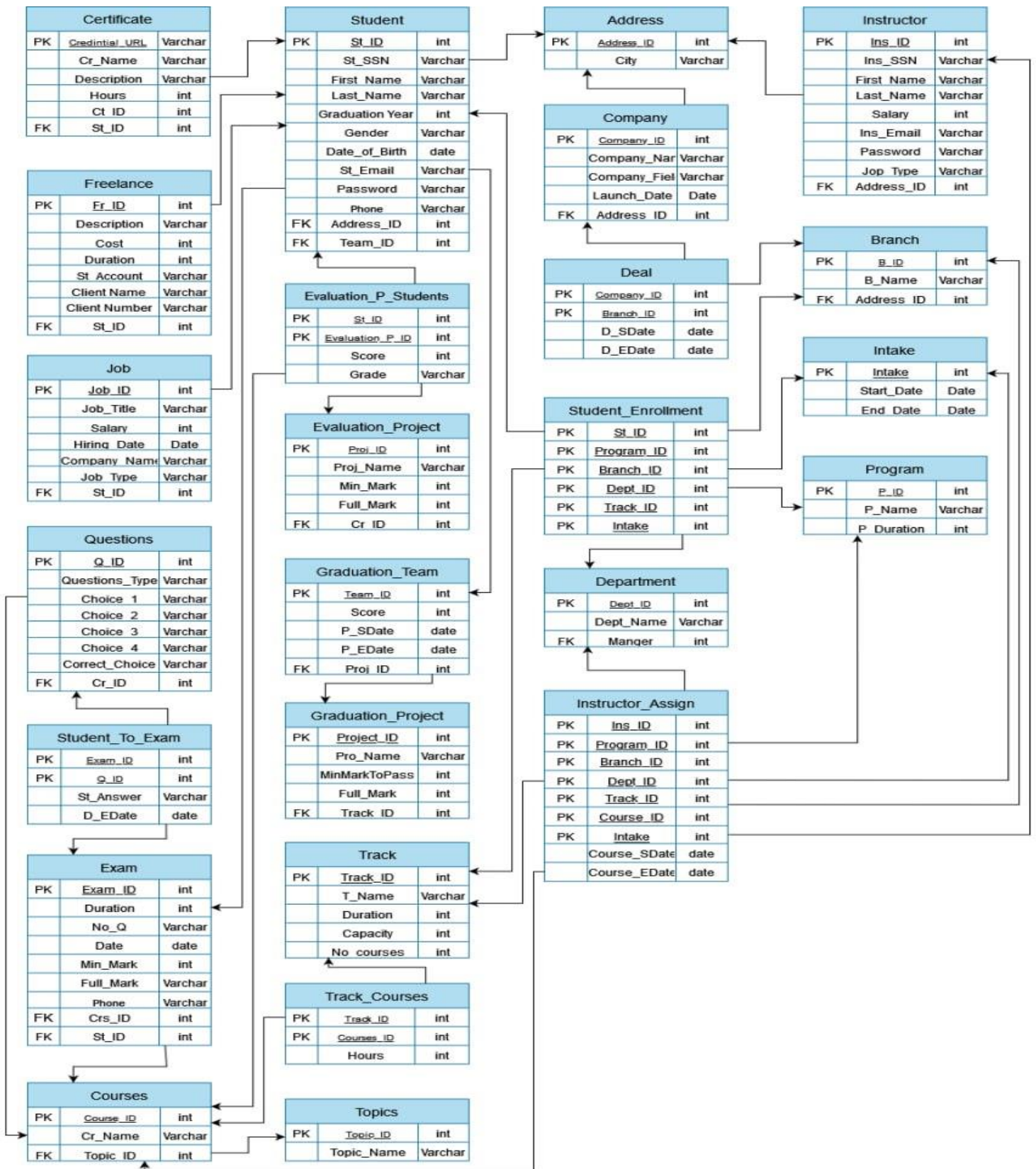


1.3 “Mapping and Normalization”

Mapping

Student	(<u>St_ID</u> ,St_SSN,First_Name,Last_Name,Graduation_Year,Date_of_Birth,St_Email>Password,Phone, Address_ID FK,Team_ID FK)
Freelance	(<u>Fr_ID</u> ,Description,Cost,Duration,St_Account,Client_Name,Client_Number,St_ID FK)
Job	(<u>Job_ID</u> ,Job_Title,Salary,Hiring_Date,Company_Name,Job_Type,St_ID FK)
Certificate	(<u>Credintial_URL</u> ,Ct_Name,Description,Hours, St_ID FK,Ct_ID FK)
Instructor	(<u>Ins_ID</u> ,Ins_SSN,First_Name,Last_Name,Salary,Date_of_Birth,Ins_Email>Password,Phone, Job_Type,Address_ID FK)
Program	(<u>P_ID</u> ,P_Name,P_Duration)
Branch	(<u>B_ID</u> ,B_Name,Address_ID FK)
Department	(<u>Dept_ID</u> ,Dept_Name,Manger FK)
Track	(<u>T_ID</u> ,T_Name,Duration,Capacity,No_Courses)
Intake	(<u>Intake</u> ,Start_Date,End_Date)
Courses	(<u>Cr_ID</u> ,Cr_Name,Topic_ID FK)
Topic	(<u>Topic_ID</u> ,Topic_Name)
Address	(<u>Address_ID</u> ,City)
Exam	(<u>Exam_ID</u> ,Duration,No_Q,Date,Min_Mark,Full_Mark,Crs_ID FK,St_ID FK)
Questions	(<u>Q_ID</u> ,Question_Type,Question,Choice_1,Choice_2,Choice_3,Choice_4,Correct_Choice,Crs_ID FK)
Company	(<u>Company_ID</u> ,Company_Name,Launch_Date,Company_Filed,Address_ID FK)
Evaluation_Projects	(<u>Proj_ID</u> ,Pro_Name,Min_Mark,Full_Mark,Cr_ID FK)
Graduation_Project	(<u>Project_ID</u> ,Pro_Name,MinMarktoPass,FullMark,Track_ID FK)
Graduation_Team	(<u>Team_ID</u> ,Score,Grade,P_SDate,P_EDate,Proj_ID FK)
Instructor_Assign	(<u>Program_ID</u> FK, <u>Branch_ID</u> FK, <u>Dept_ID</u> FK, <u>Track_ID</u> FK, <u>St_ID</u> FK, <u>Intake</u> FK)
St_Enrollment	(<u>Program_ID</u> FK, <u>Branch_ID</u> FK, <u>Dept_ID</u> FK, <u>Track_ID</u> FK, <u>Course_ID</u> FK, <u>Ins_ID</u> FK, <u>Intake</u> FK,Course_SData,Course_EDate)
Track_Courses	(<u>Cr_ID</u> FK, <u>Track_ID</u> FK,Hours)
Evaluation_P_Student	(<u>St_ID</u> FK, <u>Evaluation_P_ID</u> FK,Score,Grade)
Student_To_Exam	(<u>Exam_ID</u> FK, <u>Q_ID</u> FK,St_Answer,St_Degree,Exam_Date)

Normalization



2. Physical Implementation of Database

2.1 “Creation of Tables and Constraints”

+-----Address

```
CREATE TABLE Address (  
    Address_ID int IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    City varchar(50) NOT NULL  
);
```

-----Branch

```
CREATE TABLE Branch(  
    B_ID int IDENTITY(1,1) NOT NULL Primary Key,  
    B_Name varchar(100) NULL,  
    Address_ID int NULL,  
)
```

-----Certificate

```
CREATE TABLE Certificate (  
    Credintial_URL varchar(200) NOT NULL PRIMARY KEY,  
    Ct_Name varchar(50) NULL,  
    Description varchar(200) NULL,  
    Hours int NULL,  
    St_ID int NULL,  
    Ct_ID int IDENTITY(1,1) NOT NULL,  
    FOREIGN KEY (St_ID) REFERENCES Student (St_ID)  
);
```

-----Company

```
CREATE TABLE Company(  
    Company_ID int IDENTITY(1,1) NOT NULL primary key,  
    Company_Name varchar(100) NOT NULL,  
    Launch_Date date NOT NULL,  
    Company_Filed varchar(100) NOT NULL,  
    Address_ID int NOT NULL,  
    FOREIGN KEY(Address_ID)REFERENCES Address (Address_ID)  
)
```

-----Courses

```
CREATE TABLE Courses(  
    Cr_ID int IDENTITY(1,1) NOT NULL Primary key ,  
    Cr_Name varchar (100) NULL,  
    Topic_ID int NOT NULL,  
    FOREIGN KEY(Topic_ID)REFERENCES Topic (Topic_ID)  
)
```

-----Deal

```
CREATE TABLE Deal(  
    Branch_ID int NOT NULL,  
    Company_ID int NOT NULL,  
    D_SDate date NULL,  
    D_EDate date NULL,  
    PRIMARY KEY (Branch_ID, Company_ID),  
    FOREIGN KEY(Branch_ID)REFERENCES Branch (B_ID),  
    FOREIGN KEY(Company_ID)REFERENCES Company (Company_ID)  
)
```

-----Department

```
CREATE TABLE Department(  
    Dept_ID int IDENTITY(1,1) NOT NULL Primary Key,  
    Dept_Name varchar(100) NULL,  
    Manger int NOT NULL,  
    FOREIGN KEY(Manger) REFERENCES Instructor (Ins_ID)  
)
```

-----Evaluation_P_Student

```
CREATE TABLE Evaluation_P_Student (  
    St_ID int NOT NULL,  
    Evaluation_P_ID int NOT NULL,  
    Score int NULL,  
    Grade varchar(50) NULL,  
    PRIMARY KEY (St_ID, Evaluation_P_ID),  
    FOREIGN KEY (St_ID) REFERENCES Student (St_ID),  
    FOREIGN KEY (Evaluation_P_ID) REFERENCES Evaluation_Projects (Proj_ID)  
);
```

-----Evaluation Projects

```
CREATE TABLE Evaluation_Projects(  
    Proj_ID int IDENTITY(1,1) NOT NULL primary key,  
    Pro_Name varchar(100) NULL,  
    Min_Mark int NULL,  
    Full_Mark int NULL,  
    Cr_ID int NOT NULL,  
    FOREIGN KEY(Cr_ID) REFERENCES Courses (Cr_ID)  
)
```

-----Exam

```
CREATE TABLE Exam (  
    Exam_ID int IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Duration int NULL,  
    No_Q int NULL,  
    [Date] date NULL,  
    Min_Mark int NULL,  
    Full_Mark int NULL,  
    Crs_ID int NOT NULL,  
    St_ID int NOT NULL,  
    FOREIGN KEY (Crs_ID) REFERENCES Course (Cr_ID),  
    FOREIGN KEY (St_ID) REFERENCES Student (St_ID)  
);
```

-----Freelance

```
CREATE TABLE Freelance (  
    Fr_ID int IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Description varchar(200) NULL,  
    Cost int NULL,  
    Duration int NULL,  
    St_Account varchar(200) NULL,  
    Client_Name varchar(200) NULL,  
    Client_Number varchar(50) NULL,  
    St_ID int NULL,  
    FOREIGN KEY (St_ID) REFERENCES Student (St_ID)  
);
```

-----Graduation_Project

```
CREATE TABLE Graduation_Project(
    Project_ID int IDENTITY(1,1) NOT NULL Primary key,
    Pro_Name varchar(100) NULL,
    MinMarktoPass int NULL,
    FullMark int NULL,
    Track_ID int NULL,
    FOREIGN KEY(Track_ID)REFERENCES Track (T_ID)
)
```

-----Graduation Team

```
CREATE TABLE Graduation_Team(
    Team_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Score int NULL,
    Grade varchar(100) NULL,
    P_SDate date NULL,
    P_EDate date NULL,
    Proj_ID int NOT NULL,
    FOREIGN KEY(Proj_ID) REFERENCES Graduation_Project (Project_ID)
)
```

-----Instructor

```
CREATE TABLE Instructor(
    Ins_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Ins_SSN varchar(50) NULL,
    First_Name varchar(500) NOT NULL,
    Last_Name varchar(500) NOT NULL,
    Salary int NOT NULL,
    Ins_Email varchar(50) NOT NULL,
    Password varchar(50) NULL,
    Phone varchar(50) NOT NULL,
    Dept_ID int NOT NULL,
    Address_ID int NOT NULL,
    Job_Type varchar(100) NOT NULL,
    FOREIGN KEY(Address_ID)REFERENCES Address (Address_ID)
)
```

-----Instructor Assign

```
CREATE TABLE Instructor_Assign(
    Program_ID int NOT NULL,
    Branch_ID int NOT NULL,
    Dept_ID int NOT NULL,
    [Track_ID] int NOT NULL,
    Course_ID int NOT NULL,
    Ins_ID int NOT NULL,
    Intake int NOT NULL,
    Course_SData date NOT NULL,
    Course_EDate date NOT NULL,
    PRIMARY
KEY(Program_ID,Branch_ID,Dept_ID,Track_ID,Course_ID,Ins_ID,Intake),
    FOREIGN KEY(Branch_ID)REFERENCES Branch (B_ID),
    FOREIGN KEY(Course_ID)REFERENCES Courses (Cr_ID),
    FOREIGN KEY(Dept_ID)REFERENCES Department (Dept_ID),
    FOREIGN KEY(Ins_ID)REFERENCES Instructor (Ins_ID),
    FOREIGN KEY(Program_ID)REFERENCES Program (P_ID),
    FOREIGN KEY(Track_ID)REFERENCES Track (T_ID)
)
```

-----Intakes

```
CREATE TABLE Intakes(
    Intake int NOT NULL Primary Key,
    Start_Date date NULL,
    End_Date date NULL,
)
```

-----Job

```
CREATE TABLE Job(
    Job_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Job_Title varchar(50) NULL,
    Salary int NULL,
    Hiring_Date date NULL,
    Company_Name varchar(100) NULL,
    Job_Type varchar(50) NULL,
    St_ID int NOT NULL,
    FOREIGN KEY(St_ID)REFERENCES Student (St_ID)
)
```

-----Program

```
CREATE TABLE Program(
    P_ID int IDENTITY(1,1) NOT NULL Primary key,
    P_Name varchar(100) NULL,
    P_Duration int NULL,
)
```

-----Questions

```
CREATE TABLE Questions(
    Q_ID int IDENTITY(1,1) NOT NULL primary Key,
    Question_Type varchar(50) NULL,
    Question varchar(1000) NULL,
    Choice_1 varchar(500) NULL,
    Choice_2 varchar(500) NULL,
    Choice_3 varchar(500) NULL,
    Choice_4 varchar(500) NULL,
    Correct_Choice varchar(500) NULL,
    Crs_ID int NOT NULL,
    FOREIGN KEY(Crs_ID)REFERENCES Courses (Cr_ID)
)
```

-----St_Enrollment

```
CREATE TABLE St_Enrollment(
    Program_ID int NOT NULL,
    Branch_ID int NOT NULL,
    Dept_ID int NOT NULL,
    Track_ID int NOT NULL,
    St_ID int NOT NULL,
    Intake int NOT NULL,
    Primary Key(Program_ID,Branch_ID,Dept_ID,Track_ID,St_ID,Intake),
    FOREIGN KEY(Branch_ID)REFERENCES Branch (B_ID),
    FOREIGN KEY(Dept_ID)REFERENCES Department (Dept_ID),
    FOREIGN KEY(Intake)REFERENCES Intakes (Intake),
    FOREIGN KEY(Program_ID)REFERENCES Program (P_ID),
    FOREIGN KEY(St_ID)REFERENCES Student (St_ID),
    FOREIGN KEY([Track_ID])REFERENCES [dbo].[Track] ([T_ID])
)
```

-----Student


```

CREATE TABLE Student(
    St_ID int IDENTITY(1,1) NOT NULL Primary Key,
    St_SSN varchar(14) NOT NULL,
    First_Name varchar(100) NOT NULL,
    Last_Name varchar(100) NOT NULL,
    Graduation_Year int NOT NULL,
    Gender varchar(1) NOT NULL,
    Date_of_Birth date NOT NULL,
    St_Email varchar(50) NOT NULL,
    Password varchar(50) NULL,
    Phone varchar(20) NOT NULL,
    Address_ID int NOT NULL,
    Team_ID int NOT NULL,
    FOREIGN KEY(Address_ID)REFERENCES Address (Address_ID),
    FOREIGN KEY(Team_ID)REFERENCES Graduation_Team (Team_ID)
)

```

-----Student To Exam

```

CREATE TABLE Student_To_Exam(
    Exam_ID int NOT NULL,
    Q_ID int NOT NULL,
    St_Answer varchar(500) NULL,
    St_Degree int NULL,
    Exam_Date datetime NULL DEFAULT (getdate()),
    Primary Key(Exam_ID,Q_ID),
)

```

-----Track

```

CREATE TABLE Track(
    T_ID int IDENTITY(1,1) NOT NULL,
    T_Name varchar(100) NULL,
    Duration int NULL,
    Capacity int NULL,
    No_Courses int NULL,
)

```

-----Topic

```

CREATE TABLE Topic (
    Topic_ID int IDENTITY(1,1) NOT NULL PRIMARY KEY,
    Topic_Name varchar(100) NULL
);

```

-----Track_Courses

```

CREATE TABLE Track_Courses(
    Cr_ID int NOT NULL,
    Track_ID int NOT NULL,
    Hours int NULL,
    Primary Key(Cr_ID,Track_ID),
    FOREIGN KEY(Cr_ID)REFERENCES Courses (Cr_ID),
    FOREIGN KEY(Track_ID)REFERENCES Track (T_ID)
)

```

2.2 “Populating Sample Data”

Here is a snippet from our sample data:

Branch

```
insert into Branch (B_Name, Address_ID) values ('Alexandria', 5);
insert into Branch (B_Name, Address_ID) values ('Menofia', 6);
```

Student

```
insert into Student (St_SSN, First_Name, Last_Name, Graduation_Year,
Gender, Date_of_Birth, St_Email, Password, Phone, Address_ID, Team_ID)
values ('09766022485991', 'Amany', 'Hafez', 2022, 'F', '2004/04/08',
'vknowlton0@blogtalkradio.com', 'oC0_`DC0SKD', '207241020556', 19, 290);
```

Instructor

```
insert into Instructor (Ins_SSN, First_Name, Last_Name, Salary, Ins_Email,
Password, Phone, Dept_ID, Address_ID, Job_Type) values ('00659994004748',
'Mohamed', 'Badawi', 467815, 'wseally0@devhub.com', 'kR5,IbCZ<!W}Vde~',
'+20746771972', 7, 17, 'Contract');
```

```
insert into Instructor (Ins_SSN, First_Name, Last_Name, Salary, Ins_Email,
Password, Phone, Dept_ID, Address_ID, Job_Type) values ('98794498297315',
'Ahmed', 'Ali', 902813, 'abberger1@dropbox.com', 'dL0>olZ3cZ2/',
'+20761350400', 8, 2, 'Internship');
```

Track

```
insert into Track (T_Name, Duration, Capacity, No_Courses, Dept_ID) values
('3D Art', 7, 39, 29, 7), ('Game Programming', 5, 24, 13, 7), ('Game Art', 5, 18, 29,
7);
```

Courses

```
insert into Courses (Cr_Name, Hours, Topic_ID) values ('Introduction to
Programming', 22, 6), ('Web Development', 98, 7), ('Data Science', 96, 3);
```

Topics

```
insert into Topic (Topic_Name) values ('Artificial Intelligence'), ('Cyber
Security'), ('Data Science');
```

3. T-SQL Data Manipulation and Views Management

3.1 “Data Manipulation Language (DML) Operations”

In SQL Server, Data Manipulation Language (DML) operations are essential for managing data within tables.

These operations include inserting, updating, deleting, and querying data. They form the core of database interaction, enabling users to modify and retrieve information as needed. Through DML statements such as INSERT, UPDATE, DELETE, and SELECT, users can manipulate data according to their requirements, ensuring data integrity and consistency.

<pre>UPDATE graduation_team SET Grade = CASE WHEN Score >= 55 AND score < 65 THEN 'PASS' WHEN Score >= 65 AND score < 75 THEN 'GOOD' WHEN Score >= 75 AND score < 85 THEN 'VGGOOD' WHEN Score >= 85 THEN 'EXCELLENT' ELSE 'FAIL' -- Keep the grade unchanged for other cases END;</pre>
<pre>UPDATE Questions SET Choice_1 = 'True', Choice_2 = 'False', Choice_3 = NULL, Choice_4 = NULL WHERE Question_Type = 'true/false';</pre>
<pre>GRADUATION PROJECT UPDATE GRADUATION_PROJECT SET project_edate = DATEADD(DAY, 15, project_sdate);</pre>
<pre>SELECT [Team_ID] ,[Score] ,[Grade] ,[Proj_ID] FROM Graduation_Team</pre>

3.2 “Stored Procedures for DML Operations”

Stored Procedures are a key feature of SQL Server that allows for the encapsulation and execution of DML operations within a defined procedure. By creating stored procedures, users can streamline repetitive tasks, enhance security, and improve performance by reducing network traffic.

These procedures contain sets of SQL statements that execute specific operations, providing a reusable and efficient solution for data manipulation tasks. Whether it's inserting records, updating data, or performing complex queries, stored procedures offer a centralized and structured approach to managing DML operations.

Stored Procedures for 'Select-Update-Delete-Insert':

SP to SELECT From Student:

```
Create Procedure getStudent(@st_id int)
AS
Begin
    if exists (select * from Student where St_ID=@st_id)
        begin
            Select *
            From Student
            where St_ID=@st_id
            select 'valid st_id'
        end
    else
        select 'invalid st_id'
    END;
```

SP to Insert into Student:

```
CREATE PROCEDURE AddStudent
    @St_SSN VARCHAR(14),
    @First_Name VARCHAR(100),
    @Last_Name VARCHAR(100),
    @Graduation_Year INT,
    @Gender VARCHAR(1),
    @Date_of_Birth DATE,
    @St_Email VARCHAR(50),
    @Password VARCHAR(50),
    @Phone VARCHAR(20),
    @Address_ID INT,
    @Team_ID INT
AS
BEGIN
    BEGIN TRY
        IF EXISTS (SELECT * FROM Address WHERE Address = @Address_ID) AND
            EXISTS (SELECT * FROM Graduation_Team WHERE Team_ID = @Team_ID)
        BEGIN
            IF NOT EXISTS (SELECT * FROM Student WHERE St_SSN = @St_SSN)
            BEGIN
                INSERT INTO Student (St_SSN, First_Name, Last_Name, Graduation_Year, Gender, Date_of_Birth, St_Email, Password,
phone, Address_ID, Team_ID)
                VALUES (@St_SSN, @First_Name, @Last_Name, @Graduation_Year, @Gender, @Date_of_Birth, @St_Email, @Password,
@Phone, @Address_ID, @Team_ID);

                SELECT 'Student added successfully' AS Message;
            END
        ELSE
```

```

BEGIN
    SELECT 'Student with the provided SSN already exists' AS Message;
END
ELSE
BEGIN
    SELECT 'Please provide valid Zip Code and Team IDs' AS Message;
END
END TRY
BEGIN CATCH
    SELECT 'An error occurred. Please try again later.' AS Message;
END CATCH
END;

```

SP to Update into Student:

```

create PROCEDURE UpdateStudent
    @St_SSN VARCHAR(14),
    @First_Name VARCHAR(100) = NULL,
    @Last_Name VARCHAR(100) = NULL,
    @Graduation_Year INT = NULL,
    @Gender VARCHAR(1) = NULL,
    @Date_of_Birth DATE = NULL,
    @St_Email VARCHAR(50) = NULL,
    @Password VARCHAR(50) = NULL,
    @Phone VARCHAR(20) = NULL,
    @Address_ID INT = NULL,
    @Team_ID INT = NULL
AS
BEGIN
    BEGIN TRY
        IF EXISTS (SELECT * FROM Student WHERE St_SSN = @St_SSN)
        BEGIN DECLARE @Old_First_Name VARCHAR(100),
            @Old_Last_Name VARCHAR(100),
            @Old_Graduation_Year INT,
            @Old_Gender VARCHAR(1),
            @Old_Date_of_Birth DATE,
            @Old_St_Email VARCHAR(50),
            @Old_Password VARCHAR(50),
            @Old_Phone VARCHAR(20),
            @Old_Address_ID INT,
            @Old_Team_ID INT;

            SELECT @Old_First_Name = First_Name,
                @Old_Last_Name = Last_Name,
                @Old_Graduation_Year = Graduation_Year,
                @Old_Gender = Gender,
                @Old_Date_of_Birth = Date_of_Birth,
                @Old_St_Email = St_Email,
                @Old_Password = Password,
                @Old_Phone = Phone,
                @Old_Address_ID = Address_ID,
                @Old_Team_ID = Team_ID
            FROM Student
            WHERE St_SSN = @St_SSN;

            -- Check if provided Address ID exists
            IF (@Address_ID IS NULL OR EXISTS (SELECT * FROM Address WHERE Address_ID = @Address_ID))
            BEGIN
                -- Check if provided Team ID exists

```

```

IF (@Team_ID IS NULL OR EXISTS (SELECT * FROM Graduation_Team WHERE Team_ID = @Team_ID))
BEGIN
    UPDATE Student
    SET First_Name = COALESCE(@First_Name, @Old_First_Name),
        Last_Name = COALESCE(@Last_Name, @Old_Last_Name),
        Graduation_Year = COALESCE(@Graduation_Year, @Old_Graduation_Year),
        Gender = COALESCE(@Gender, @Old_Gender),
        Date_of_Birth = COALESCE(@Date_of_Birth, @Old_Date_of_Birth),
        St_Email = COALESCE(@St_Email, @Old_St_Email),
        Password = COALESCE(@Password, @Old_Password),
        Phone = COALESCE(@Phone, @Old_Phone),
        Address_ID = COALESCE(@Address_ID, @Old_Address_ID),
        Team_ID = COALESCE(@Team_ID, @Old_Team_ID)
    WHERE St_SSN = @St_SSN;

    SELECT 'Student information updated successfully' AS Message;
    END
    ELSE
    BEGIN
        SELECT 'Team with the provided ID does not exist' AS Message;
        END
    END
    ELSE
    BEGIN
        SELECT 'Zip Code with the provided ID does not exist' AS Message;
        END
    END
    ELSE
    BEGIN
        SELECT 'Student with the provided SSN does not exist' AS Message;
        END
    END TRY
    BEGIN CATCH
        SELECT 'An error occurred. Please try again later.' AS Message;
    END CATCH
END;

```

SP to Delete into Student:

```

CREATE PROCEDURE DeleteStudent
    @St_SSN VARCHAR(14)
AS
BEGIN
    BEGIN TRY
        IF EXISTS (SELECT * FROM Student WHERE St_SSN = @St_SSN)
        BEGIN
            DELETE FROM Student WHERE St_SSN = @St_SSN;
            SELECT 'Student deleted successfully' AS Message;
        END
        ELSE
        BEGIN
            SELECT 'Student with the provided SSN does not exist' AS Message;
        END
    END TRY
    BEGIN CATCH
        SELECT 'An error occurred. Please try again later.' AS Message;
    END CATCH
END

```


Stored Procedures for Generating Exam:

Exams_model_SP is typically used for querying exam details and displaying them to users.

```
Create PROCEDURE [dbo].[Exams_model_SP]
    @studentID INT,
    @courseID INT
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Exam AS e WHERE e.St_ID = @studentID AND e.Crs_ID = @courseID)
    BEGIN
        IF EXISTS (SELECT 1 FROM Student_To_Exam AS ste INNER JOIN Exam AS e ON ste.Exam_ID = e.Exam_ID
            WHERE e.St_ID = @studentID AND e.Crs_ID = @courseID)
        BEGIN
            SELECT e.Exam_ID, q.Question, q.Correct_Choice, ste.St_Answer, ste.St_Degree
            FROM Questions AS q
            INNER JOIN Student_To_Exam AS ste ON q.Q_ID = ste.Q_ID
            INNER JOIN Exam AS e ON ste.Exam_ID = e.Exam_ID
            WHERE e.St_ID = @studentID AND e.Crs_ID = @courseID;
        END
    END
    ELSE
    BEGIN
        SELECT 'this student has not taken the exam for this course yet!!' AS message;
    END
    ELSE
    BEGIN
        SELECT 'Please enter valid IDs!!' AS message;
    END
END
GO
```

GenerateExam SP is invoked when administrators or system processes need to create exams for a specific set of students based on certain criteria.

```
create PROCEDURE GenerateExam
    @Intake_ID int,
    @Track_ID int,
    @Crs_ID int,
    @Date date
AS
BEGIN
    -- Check if Intake_ID exists
    IF EXISTS (SELECT * FROM Intakes WHERE Intake = @Intake_ID)
    BEGIN
        -- Check if Track_ID exists
        IF EXISTS (SELECT * FROM Track WHERE T_ID = @Track_ID)
        BEGIN
            -- Check if Crs_ID exists
            IF EXISTS (SELECT * FROM Courses WHERE Cr_ID = @Crs_ID)
            BEGIN
                -- Check if there are enrollments for the specified Intake, Track, and Course
                IF EXISTS (SELECT se.Intake, se.Track_ID, tc.Cr_ID FROM St_Enrollment se
                    JOIN Track_Courses tc ON tc.Track_ID = se.Track_ID
                    WHERE se.Intake = @Intake_ID AND se.Track_ID = @Track_ID AND tc.Cr_ID = @Crs_ID)
                BEGIN
                    -- Check if there are no existing exams for the specified date, course, and students
                    IF NOT EXISTS (SELECT * FROM Exam WHERE Date = @Date AND Crs_ID = @Crs_ID AND St_ID IN (SELECT
```

```

St_ID FROM St_Enrollment WHERE Intake = @Intake_ID AND Track_ID = @Track_ID))
BEGIN
    -- Insert exams for each student enrolled in the specified Intake, Track, and Course
    INSERT INTO Exam (Date, Crs_ID, St_ID)
    SELECT @Date, @Crs_ID, se.St_ID
    FROM St_Enrollment se
    JOIN Track_Courses tc ON tc.Track_ID = se.Track_ID
    WHERE se.Intake = @Intake_ID AND se.Track_ID = @Track_ID AND tc.Cr_ID = @Crs_ID;

    PRINT 'Exams generated successfully.';
END
ELSE
BEGIN
    PRINT 'Exams already exist for the specified date, course, and students.';
END
END
ELSE
BEGIN
    PRINT 'No enrollments found for the specified Intake, Track, and Course.';
END
END
ELSE
BEGIN
    PRINT 'Course with ID ' + CAST(@Crs_ID AS varchar(10)) + ' does not exist.';
END
END
ELSE
BEGIN
    PRINT 'Track with ID ' + CAST(@Track_ID AS varchar(10)) + ' does not exist.';
END
END
ELSE
BEGIN
    PRINT 'Intake with ID ' + CAST(@Intake_ID AS varchar(10)) + ' does not exist.';
END
END
END

```

Stored Procedures for Exam “Answer-Correction-Degree”:

Exam Answer SP:

```

CREATE PROCEDURE ExamAnswers
    @Exam_ID INT
AS
BEGIN
    -- Check if the provided Exam_ID exists in the Exam table
    IF NOT EXISTS (SELECT 1 FROM Exam WHERE Exam_ID = @Exam_ID)
    BEGIN
        Select('The provided Exam_ID does not exist in the Exam table.');
```

Exam Correction SP:

```
Create PROCEDURE ExamCorrection
    @Exam_ID INT
AS
BEGIN
    -- Check if the provided Exam_ID exists in the Exam table
    IF NOT EXISTS (SELECT 1 FROM Exam WHERE Exam_ID = @Exam_ID)
    BEGIN
        Select ('The provided Exam_ID does not exist in the Exam table.');
```

END;

-- Select data for exam correction

```
SELECT e.Exam_ID, e.St_ID, ste.Q_ID, ste.St_Answer, q.Correct_Choice, ste.St_Degree
FROM Exam e
JOIN Student_To_Exam ste ON ste.Exam_ID = e.Exam_ID
JOIN Questions q ON q.Q_ID = ste.Q_ID
WHERE e.Exam_ID = @Exam_ID;
END;
```

Exam Degree SP:

```
Create PROCEDURE ExamDegree
    @Exam_ID INT,
    @St_ID INT,
    @Cr_ID INT
AS
BEGIN
    -- Check if the provided St_ID exists in the Student table
    IF NOT EXISTS (SELECT 1 FROM Student WHERE St_ID = @St_ID)
    BEGIN
        Print('The provided St_ID does not exist in the Student table.');
```

END;

-- Check if the provided Cr_ID exists in the Courses table

```
IF NOT EXISTS (SELECT 1 FROM Courses WHERE Cr_ID = @Cr_ID)
BEGIN
    Print('The provided Cr_ID does not exist in the Courses table.');
```

END;

-- Check if the provided Exam_ID exists in the Exam table

```
IF NOT EXISTS (SELECT 1 FROM Exam WHERE Exam_ID = @Exam_ID)
BEGIN
    Print('The provided Exam_ID does not exist in the Exam table.');
```

END;

-- Select data

```
SELECT e.Exam_ID,
    s.St_ID,
    CONCAT_WS(' ', s.First_Name, s.Last_Name) AS 'Student Name',
    c.Cr_Name,
    SUM(ste.St_Degree) AS [student degree],
    MAX(ste.Exam_Date) AS [Submit Date]
FROM Exam AS e
INNER JOIN Student_To_Exam AS ste ON e.Exam_ID = ste.Exam_ID
JOIN Student AS s ON s.St_ID = e.St_ID
JOIN Courses AS c ON c.Cr_ID = e.Crs_ID
WHERE e.Exam_ID = @Exam_ID AND e.Crs_ID = @Cr_ID
GROUP BY s.St_ID, CONCAT_WS(' ', s.First_Name, s.Last_Name), c.Cr_Name, e.Exam_ID;
END;
```

3.3 “Views Creation”

Views in SQL Server offer a dynamic and virtual representation of data stored in tables. They provide a way to simplify complex queries, enhance data security, and improve performance by presenting a tailored perspective of the underlying data. Views are created using SELECT statements, enabling users to define customized data sets based on specific criteria. By encapsulating complex joins, calculations, and filters, views offer a simplified interface for users to interact with the database, abstracting the underlying complexity of the data model.

"Exams_model" View: is useful for analyzing exam performance, identifying patterns, and gaining a comprehensive understanding of student responses within the context of specific exams.

```
Create view Exams_model
as
select e.Exam_ID,e.St_ID,e.Crs_ID,q.Question,q.Correct_Choice, ste.St_Answer,ste.St_Degree
from Questions as q
inner join Student_To_Exam as ste
on q.Q_ID = ste.Q_ID
inner join Exam as e
on ste.Exam_ID=e.Exam_ID
```

"St_Crs_Ins" View: This view facilitates analysis of student-course assignments, instructor allocations, and program enrollments, aiding in academic planning, monitoring, and assessment.

```
create view [dbo].[St_Crs_Ins]
as
select s.St_ID,Concat(S.First_Name,' ',S.Last_Name) as 'Student Name' , p.P_Name as 'Program Name',b.B_Name
as 'Branch Name',
d.Dept_Name as 'Department Name',t.T_Name as 'Track Name',c.Cr_Name as 'Course
Name',ia.Ins_ID,Concat(ia.First_Name,' ',ia.Last_Name) as 'Instructor Name',i.*
from St_Enrollment as ste
inner join Track_Courses as tc
on ste.Track_ID = tc.Track_ID
inner join Instructor_Assign as ia
on ia.Program_ID = ste.Program_ID
and
ia.Branch_ID = ste.Branch_ID
and
ia.Track_ID = ste.Track_ID
and
ia.Course_ID = tc.Cr_ID
and
ia.Intake = ste.Intake
join Program p
on p.P_ID=ste.Program_ID
join Branch b
on b.B_ID=ste.Branch_ID
join Department d
on d.Dept_ID=ste.Dept_ID
Join Track t
on t.T_ID=ste.Track_ID
```

```

join student s
on s.st_ID=ste.St_ID
join Courses c
on c.Cr_ID=tc.Cr_ID
join Intakes i
on i.Intake=ste.Intake
inner join Instructor as ins
on ins.Ins_ID = ia.Ins_ID

```

"std" view: simulates a student's exam experience by providing randomly generated answers to questions and calculating a score based on correctness. This can be useful for testing and evaluating exam systems or for generating practice exams.

```

create view St_Crs_Ins
as
select s.St_ID,Concat(S.First_Name,' ',S.Last_Name) as 'Student Name' , p.P_Name as 'Program Name',b.B_Name
as 'Branch Name',
d.Dept_Name as 'Department Name',t.T_Name as 'Track Name',c.Cr_Name as 'Course
Name',ia.Ins_ID,Concat(ins.First_Name,' ',ins.Last_Name) as 'Instructor Name',i.*
from St_Enrollment as ste
inner join Track_Courses as tc
on ste.Track_ID = tc.Track_ID
inner join Instructor_Assign as ia
on ia.Program_ID = ste.Program_ID
and
ia.Branch_ID = ste.Branch_ID
and
ia.Track_ID = ste.Track_ID
and
ia.Course_ID = tc.Cr_ID
and
ia.Intake = ste.Intake
join Program p
on p.P_ID=ste.Program_ID
join Branch b
on b.B_ID=ste.Branch_ID
join Department d
on d.Dept_ID=ste.Dept_ID
Join Track t
on t.T_ID=ste.Track_ID
join student s
on s.st_ID=ste.St_ID
join Courses c
on c.Cr_ID=tc.Cr_ID
join Intakes i
on i.Intake=ste.Intake
inner join Instructor as ins
on ins.Ins_ID = ia.Ins_ID

```

"students_Exams" view: The view groups the data by exam ID, student ID, student name, course name, program name, branch name, track name, intake, minimum mark, and full mark. This provides a concise summary of exam performance for each student across different courses and programs.

```
CREATE view students_Exams
as
select ste.Exam_ID,se.St_ID,CONCAT_WS(' ', s.First_Name, s.Last_Name) AS 'Student Name',c.Cr_Name,p.P_Name,b.B_Name,
t.T_Name,se.Intake,e.Min_Mark,e.Full_Mark,Sum(ste.St_Degree) as [student degree],max(ste.Exam_Date) as [Submit Date]
from St_Enrollment as se
inner join Student s
on se.St_ID = s.St_ID
inner join Program as p
on se.Program_ID = p.P_ID
inner join Branch as b
on se.Branch_ID = b.B_ID
inner join Track as t
on se.Track_ID = t.T_ID
inner join Track_Courses as tc
on se.Track_ID = tc.Track_ID
inner join Courses as c
on tc.Cr_ID = c.Cr_ID
inner join Exam as e
on e.St_ID = se.St_ID and e.Crs_ID = tc.Cr_ID
inner join Student_To_Exam as ste
on e.Exam_ID = ste.Exam_ID
group by ste.Exam_ID,se.St_ID,CONCAT_WS(' ', s.First_Name,
s.Last_Name),c.Cr_Name,p.P_Name,b.B_Name,t.T_Name,se.Intake,e.Min_Mark,e.Full_Mark
```

"yetExamEd" view: is designed to display exams that have not yet been taken by any student.

```
create view yetExamEd
as
select *
from Exam
where Exam_ID NOT IN (SELECT ste.Exam_ID FROM Student_To_Exam AS ste)
```

4. REPORTING

4.1 & 4.2 “Reporting Operations: Stored Procedures and Visual Reporting”

1. Report that returns the students information according to Department No parameter.

The report stored procedure & Preview:

```
create Procedure GetDeptStd @Dept_ID VARCHAR(MAX)
AS
BEGIN
IF EXISTS (SELECT 1 FROM Department WHERE Dept_ID IN (SELECT value FROM STRING_SPLIT(@Dept_ID, ',')))
BEGIN
SELECT
s.St_ID,
s.St_SSN,
CONCAT_WS(' ', s.First_Name, s.Last_Name) AS 'Full Name',
s.Gender,
s.St_Email,
s.Date_of_Birth,
s.Graduation_Year,
s.Phone,
a.City,
gt.Team_ID,
gt.Score,
gt.P_SDate,
gt.P_EDate
FROM
student s
JOIN
St_Enrollment se ON se.St_ID = s.St_ID
JOIN
Address a ON a.Address_ID = s.Address_ID
JOIN
Graduation_Team gt ON gt.Team_ID = s.Team_ID
WHERE
Dept_ID IN (SELECT value FROM STRING_SPLIT(@Dept_ID, ','))
END
ELSE
SELECT 'Department ID Does not Exist' AS Message
END
```

 SQL Server Reporting Services



★ Favorites  Browse  Comments

Home > Report Project2 > StudentInformation

Dept ID 







Students Information per Department

Dept_ID : 7 Department Name: Content Developments

St ID	St SSN	Full Name	Gender	St Email	Date of Birth	Graduation Year	Phone	City	Team ID	Score	Project Start Date	Project End Date
1	09766022485991	Amany Hafez	F	vkowit0n0@biogtalkradio.com	04/08/2004	2022	207241020556	Fayoum	1	94	01/01/2015	01/16/2015
2	45724695053316	Marwa Sobhy	F	gwarcop1@usda.gov	06/22/1994	2023	209975909767	AlSharqia	1	94	01/01/2015	01/16/2015
3	14758238396151	El-Sayed Mostafa	M	sstafford2@bing.com	02/26/2002	2029	204012507058	Cairo	1	94	01/01/2015	01/16/2015
4	66681058194607	Ibrahim Mansour	F	dlutsch3@baidu.com	07/19/1999	2018	208835940058	Sohag	1	94	01/01/2015	01/16/2015
5	75354525777444	Farah Amin	M	gketchen4@is.gd	03/14/1999	2016	202832287994	Giza	1	94	01/01/2015	01/16/2015
6	06709123152809	Ghada Badawi	F	obarber5@prnewswire.com	11/05/1990	2030	206737991028	South Sinai	2	97	01/01/2015	01/16/2015
7	33515304514804	Aliaa Shehata	F	blutchings5@gmail.com	02/13/1994	2017	202874315157	Luxor	2	97	01/01/2015	01/16/2015
8	29461222516499	Fahmy Gad	F	dwheeliker7@time.com	04/25/1994	2023	201233695698	New Valley	2	97	01/01/2015	01/16/2015
9	88559701757828	Eman Hassan	F	mborthwick8@usgs.gov	03/19/1995	2018	209916819808	Suez	2	97	01/01/2015	01/16/2015
10	36557224285040	Farah El-Gendy	F	dfglovann9@eepurl.com	09/06/2005	2017	207394311009	North Sinai	2	97	01/01/2015	01/16/2015
11	50350105021513	Farah Abdel	F	jhasleya@timesonline.co.uk	06/27/1992	2025	208508475588	Luxor	3	46	01/01/2015	01/16/2015
12	96239072455441	Salem Khalil	M	rijandourek@issuu.com	03/19/2004	2016	201450752203	Alexandria	3	46	01/01/2015	01/16/2015
13	06132520715424	Mahmoud Aziz	M	bkualekt@hao123.com	05/15/1991	2027	205722560204	Mansoura	3	46	01/01/2015	01/16/2015
14	69281609491699	Marwa El-Zaher	F	nrolis@toplist.cz	10/19/2002	2023	201805081761	New Valley	3	46	01/01/2015	01/16/2015
15	43828245308772	Tawfik Nasser	F	gdoorowe@dailymotion.com	08/17/1990	2011	205721527724	Menofia	3	46	01/01/2015	01/16/2015
16	81776200673502	Eman Nasser	M	gsmillie@webmd.com	05/17/1998	2015	205845536786	Sohag	4	48	01/01/2015	01/16/2015
17	40632679503490	Amin El-Nour	F	jgentille@jimdo.com	11/03/1994	2017	206220703680	Bane Suef	4	48	01/01/2015	01/16/2015
18	72980059448877	Sohila Mostafa	F	abosseleh@comsenz.com	12/18/2002	2024	201028078121	Bane Suef	4	48	01/01/2015	01/16/2015
19	06831143687492	Tawfik El-Amir	M	haimel@prnewswire.com	10/16/1995	2014	203613628368	Red Sea	4	48	01/01/2015	01/16/2015
..			..						..	..		

Activate Windows
Go to Settings to activate Windows.

- Report that takes the student ID and returns the grades of the student in all courses. %

The report stored procedure & Preview:

```

Create procedure GetStdDefgree @St_ID VARCHAR(MAX)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Student WHERE St_ID IN (SELECT value FROM STRING_SPLIT(@St_ID, ',')))
    BEGIN
        IF EXISTS (SELECT 1 FROM St_Enrollment WHERE St_ID IN (SELECT value FROM STRING_SPLIT(@St_ID, ',')))
        BEGIN
            SELECT
                se.St_ID,
                Crs_ID,
                c.Cr_Name,
                e.Full_Mark,
                ((SUM(st.St_Degree)) * 100 / e.Full_Mark) AS ' % Degree'
            FROM
                St_Enrollment se
            JOIN
                Exam e ON e.St_ID = se.St_ID
            JOIN
                Student_To_Exam ste ON ste.Exam_ID = e.Exam_ID
            JOIN
                Courses c ON c.Crs_ID = e.Crs_ID
            WHERE
                se.St_ID IN (SELECT value FROM STRING_SPLIT(@St_ID, ','))
            GROUP BY
                se.St_ID,
                Crs_ID,
                c.Cr_Name,
                e.Full_Mark
        END
    ELSE
        SELECT 'Student ID Does not Exist in St_Enrollment' AS Message
    END
    ELSE
        SELECT 'Student ID Does not Exist' AS Message
    END
END

```

SQL Server Reporting Services

★ Favorites 🔍 Browse

[Home](#) > [Report Project2](#) > StdDegree

St_ID View Report

1 of 1 Page Width Find | Next



Students Degrees Report

St_ID: 167 & St_Name :Hadeer Badawi

St ID	Crs ID	Cr Name	Degree%	Full Mark	St_Indicator
167	10	Operation system	50 %	20	🚩
167	11	XML	20 %	20	🚩
167	12	SQL BI	80 %	20	🟢
167	13	Excel	40 %	20	🚩
167	24	Communications skills	0 %	20	🚩

Activate Windows
Go to Settings to activate Windows.

- Report that takes the instructor ID and returns the name of the courses that he teaches and the number of students per course.

The report stored procedure & Preview:

```
Create PROCEDURE TeachingIns
    @Ins_ID VARCHAR(MAX)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Instructor WHERE Ins_ID IN (SELECT value FROM STRING_SPLIT(@Ins_ID, ',')))
    BEGIN
        IF EXISTS (SELECT 1 FROM Instructor_Assign WHERE Ins_ID IN (SELECT value FROM STRING_SPLIT(@Ins_ID, ',')))
        BEGIN
            SELECT
                COUNT(s.St_ID) AS 'No_Student',
                p.P_Name AS 'Program Name',
                b.B_Name AS 'Branch Name',
                d.Dept_Name AS 'Department Name',
                t.T_Name AS 'Track Name',
                c.Cr_Name AS 'Course Name',
                ia.Ins_ID,
                CONCAT(ins.First_Name, ' ', ins.Last_Name) AS 'Instructor Name',
                i.*
            FROM
                St_Enrollment AS ste
            INNER JOIN
                Track_Courses AS tc ON ste.Track_ID = tc.Track_ID
            INNER JOIN
                Instructor_Assign AS ia ON ia.Program_ID = ste.Program_ID
                    AND ia.Branch_ID = ste.Branch_ID
                    AND ia.Track_ID = ste.Track_ID
                    AND ia.Course_ID = tc.Cr_ID
                    AND ia.Intake = ste.Intake

            JOIN
                Program p ON p.P_ID = ste.Program_ID
            JOIN
                Branch b ON b.B_ID = ste.Branch_ID
            JOIN
                Department d ON d.Dept_ID = ste.Dept_ID
            JOIN
                Track t ON t.T_ID = ste.Track_ID
            JOIN
                Student s ON s.st_ID = ste.St_ID
            JOIN
                Courses c ON c.Cr_ID = tc.Cr_ID
            JOIN
                Intakes i ON i.Intake = ste.Intake
            INNER JOIN
                Instructor AS ins ON ins.Ins_ID = ia.Ins_ID
            WHERE
                ins.Ins_ID IN (SELECT value FROM STRING_SPLIT(@Ins_ID, ','))
            GROUP BY
                p.P_Name,
                b.B_Name,
                d.Dept_Name,
                t.T_Name,
                c.Cr_Name,
                ia.Ins_ID,
                CONCAT(ins.First_Name, ' ', ins.Last_Name),
                i.Intake,
                i.Start_Date,
                i.End_Date
        END
    ELSE
        SELECT 'Instructor ID Does not Assign to any Courses' AS Message
    END
    ELSE
        SELECT 'Instructor ID Does not Exist' AS Message
    END
END
```

SQL Server Reporting Services

★ Favorites
Browse
Comments

Home > Report Project2 > InsTeaching

Ins ID
View Report

1 of 1
Whole Page
Find | Next

ITI Instructor's Courses Report Ins ID : 1 , 2 , 3 Ins Name: Mohamed Badawi , Ahmed Ali , Ahmed Adel											
Ins ID	Instructor Name	Course Name	Track Name	Branch Name	Program Name	Department Name	Intake	Start Date	End Date	No Student	
1	Mohamed Badawi	Data Warehouse	AI and Machine Learning Engineer	Minya	Early Career Build Up - (Summer Training)	Cognitive Computing And Artificial Intelligence	111	7/1/2022	8/1/2022	20	
2	Ahmed Ali	Advanced Data base	Enterprise & Web Apps Development - Java	Ismailia	Intensive Code Camp - (ICC)	Software Engineering And Development	68	7/1/2022	10/1/2023	15	
1	Mohamed Badawi	Data Science	Geographic information Systems	Minya	Professional Training Program - (9 Months)	Information Systems	32	10/1/2020	7/1/2021	10	
2	Ahmed Ali	Power BI	Geographic information Systems	Minya	Professional Training Program - (9 Months)	Information Systems	32	10/1/2020	7/1/2021	10	
3	Ahmed Adel	Data base Fundamental	Data Management	Smart Village	Professional Training Program - (9 Months)	Information Systems	34	10/1/2022	7/1/2023	10	
2	Ahmed Ali	Mobile App Development	Data Management	Smart Village	Professional Training Program - (9 Months)	Information Systems	35	10/1/2023	7/1/2024	10	
1	Mohamed Badawi	Machine Learning	Data Science	Smart Village	Professional Training Program - (9 Months)	Information Systems	28	10/1/2016	7/1/2017	10	
2	Ahmed Ali	Machine Learning	Data Science	Smart Village	Professional Training Program - (9 Months)	Information Systems	32	10/1/2020	7/1/2021	10	

4. Report that takes course ID and returns its topics.

The report stored procedure & Preview:

```
Create PROCEDURE CoursesTopic
@Cr_ID VARCHAR(MAX) – Change data type to VARCHAR(MAX) to handle multiple values
AS
BEGIN
IF EXISTS (SELECT 1 FROM Courses WHERE Cr_ID IN (SELECT value FROM STRING_SPLIT(@Cr_ID, ',')))
BEGIN
SELECT
C.Cr_ID,
t.*
FROM
Courses c
JOIN
Topic t ON t.Topic_ID = c.Topic_ID
WHERE
c.Cr_ID IN (SELECT value FROM STRING_SPLIT(@Cr_ID, ','))
END
ELSE
SELECT 'Course ID Does not Exist' AS Message
END
```

SQL Server Reporting Services

★ Favorites
Browse

Home > Report Project2 > CourseTopic

Cr ID

1 of 1
100%

ITI

Course's Topics

Course_ID: 2 , 3 , 4 , 5 , 6

Cr ID	Topic ID	Topic Name
2	7	web
3	3	Data Science
4	1	Artificial Intelligence
5	2	Cyber Security
6	11	Mobile Development

5. Report that takes exam number and returns the Questions in it and choices [freeform report].

The report stored procedure & Preview:

```
create PROCEDURE GetExam
    @Exam_ID VARCHAR(MAX)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Exam WHERE Exam_ID IN (SELECT value FROM STRING_SPLIT(@Exam_ID, ',')))
    BEGIN
        SELECT
            e.Exam_ID,
            ste.Q_ID,
            q.Question,
            q.Choice_1,
            q.Choice_2,
            q.Choice_3,
            q.Choice_4
        FROM
            Exam e
        JOIN
            Student_To_Exam ste ON ste.Exam_ID = e.Exam_ID
        JOIN
            Questions q ON q.Q_ID = ste.Q_ID
        WHERE
            e.Exam_ID IN (SELECT value FROM STRING_SPLIT(@Exam_ID, ','))
    END
    ELSE
        SELECT 'Exam ID Does not Exist' AS Message
    END
```

SQL Server Reporting Services

★ Favorites

🔍 Browse

Comments

Home > Report Project2 > Exam

Exam ID ▼

View Report

⏪ < 1 of 1 > ⏩ ↺ ⌂

Whole Page

📄

Find | Next



Exam Questions

Exam ID: 3

Exam ID	Q ID	Question	Choice 1	Choice 2	Choice 3	Choice 4
3	231	Which symbol is used to denote the start and end of an XML element?	<>	[]	[]	<>
3	232	In XML, what is the purpose of an attribute?	To define the structure of an element	To store text data within an element	To provide additional information about an element	To specify the ordering of child elements
3	235	What is the purpose of CDATA in XML?	To define a comment	To escape special characters	To define a processing instruction	To include character data that should not be parsed
3	236	Which XML schema language is commonly used for defining the structure of XML documents?	DTD (Document Type Definition)	XML Schema	XSL (eXtensible Stylesheet Language)	XPath (XML Path Language)
3	238	Which of the following is NOT a valid XML namespace declaration syntax?	xmlns="http://example.com"	xmlns:prefix="http://example.com"	namespace="http://example.com"	xmlns:xx="http://example.com"
3	240	XML documents must have a root element.	True	False		
3	241	XML is case-sensitive.	True	False		
3	243	XML attributes must always have a value enclosed in quotation marks.	True	False		
3	244	XML elements can be nested to any depth.	True	False		
3	246	XML namespaces are used to avoid element name conflicts in XML documents.	True	False		

6. Report that takes exam number and the student ID then returns the Questions in this exam with the student answers.

The report stored procedure & Preview:

```
Create PROCEDURE GetStdExam
    @Exam_ID INT,
    @St_ID INT
AS
BEGIN
    IF EXISTS (
        SELECT *
        FROM Exam e
        JOIN St_Enrollment se ON se.St_ID = e.St_ID
        WHERE e.Exam_ID = @Exam_ID AND se.St_ID = @St_ID
    )
    BEGIN
        SELECT
            e.Exam_ID,e.St_ID,
            ste.Q_ID,
            q.Question,
            ste.St_Answer,q.Correct_Choice,ste.St_Degree
        FROM
            Exam e
        JOIN
            Student_To_Exam ste ON ste.Exam_ID = e.Exam_ID
        JOIN
            Questions q ON q.Q_ID = ste.Q_ID
        JOIN
            St_Enrollment se ON se.St_ID = e.St_ID
        WHERE
            e.Exam_ID = @Exam_ID AND se.St_ID = @St_ID;
    END
    ELSE
    BEGIN
        PRINT 'No such exam found for the given student.';
    END
END;
```


SQL Server Reporting Services

★ Favorites □ Browse

[Home](#) > [Report Project2](#) > StdExam

Exam ID St_ID

<| < 1 of 1 > >| ↺ ↻ Whole Page ▾  Find | Next



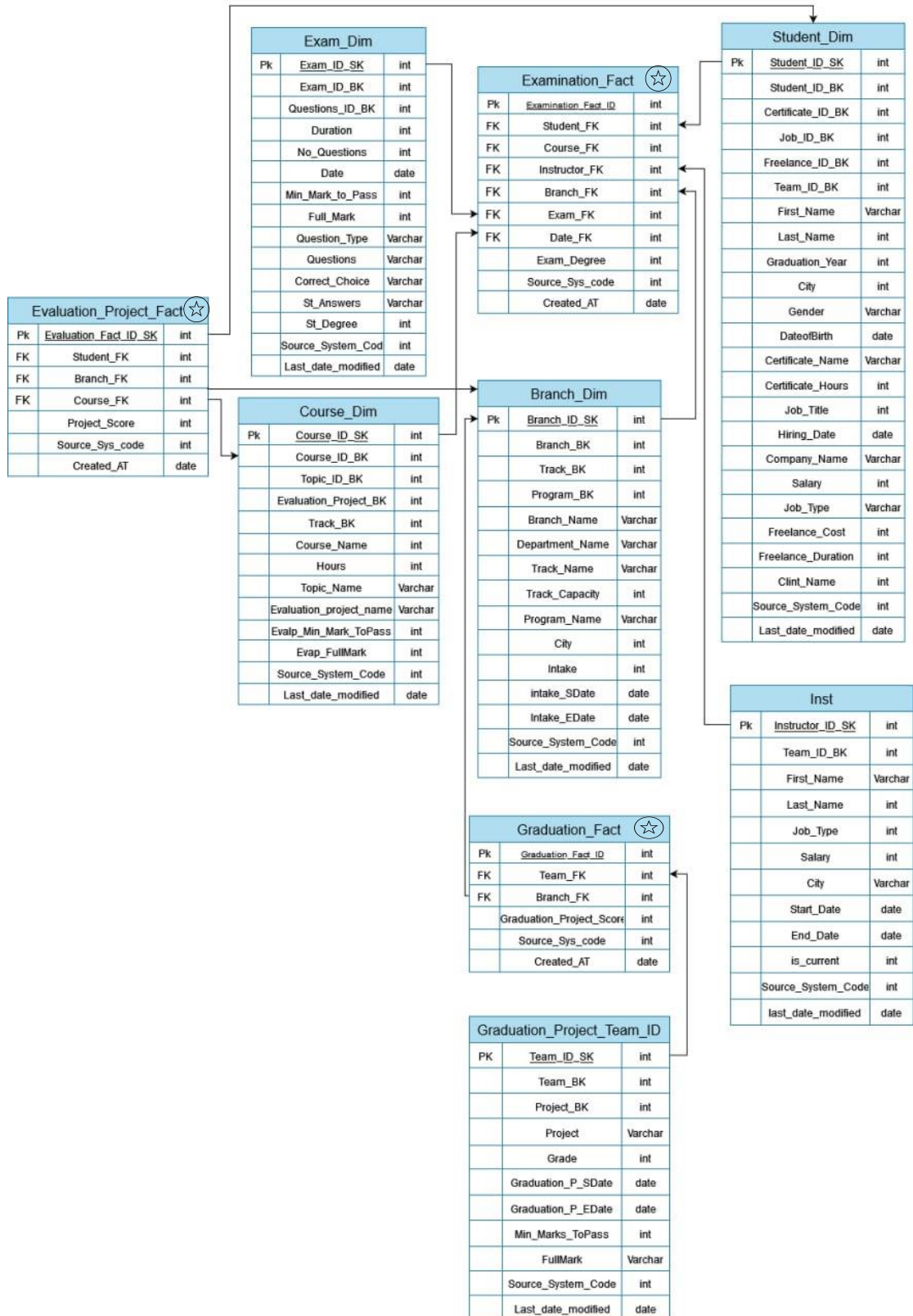
Student's Exam

St_ID: 1 Exam_ID: 2

St ID	Exam ID	Q ID	Question	Correct Choice	Student Answer	Student Degree
1	2	210	Which of the following is NOT an essential component of an operating system?	Compiler	File System	0
1	2	211	What is the main function of the operating system's kernel?	To manage system resources and provide essential services	To execute application software	0
1	2	217	What is a deadlock in the context of operating systems?	A situation where two or more processes are waiting indefinitely for a resource held by each other	A situation where a process consumes excessive CPU time	0
1	2	218	Which file system feature helps prevent data loss in the event of a system crash by ensuring that updates to critical filesystem structures are written to disk before proceeding with other operations?	Journaling	RAID	0
1	2	221	Fragmentation occurs when free memory becomes divided into small, non-contiguous blocks over time.	True	True	2
1	2	223	A shell is a user interface that allows users to interact with the operating system through a command-line interface only.	False	True	0
1	2	224	A mutex (mutual exclusion) is a synchronization primitive used to protect shared resources from being accessed simultaneously by multiple processes.	True	True	2
1	2	225	Time-sharing operating systems allow multiple users to share the same hardware resources by providing each user with a dedicated processor.	False	True	0
1	2	226	A semaphore is a synchronization primitive that can only take two values: 0 and 1.	False	True	0
1	2	227	A page fault occurs when a requested page is not present in main memory and must be retrieved from secondary storage.	True	True	2

5. DataWarehouse

5.1 “Design Overview: multidimensional data modelling”



5.2 “Implementing the Data Warehouse”

```
-----Branch Dimension
CREATE TABLE Branches_DIM(
    Branch_ID int IDENTITY(1,1) NOT NULL Primary key,
    Branch_BK int NOT NULL,
    B_City varchar(50) NULL,
    Department_BK int NOT NULL,
    Track_BK int NOT NULL,
    Program_BK int NOT NULL,
    Branch_Name varchar(100) NULL,
    Department_Name varchar(100) NULL,
    Track_Name varchar(100) NULL,
    Track_Capacity int NULL,
    Program_Name varchar(200) NULL,
    Intake int NULL,
    intake_SDate date NULL,
    Intake_EDate date NULL,
    Source_SYS_Code tinyint NULL,
    Last_Modified_Date date NULL DEFAULT (getdate())
)

-----Course Dimension
CREATE TABLE Course_Dim(
    Course_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Course_BK int NULL,
    Topic_BK int NULL,
    Evaluation_Project_BK int NULL,
    Course_Name varchar(200) NULL,
    Track_BK int NULL,
    Hours int NULL,
    Topic_Name varchar(200) NULL,
    Evaluation_project_name varchar(200) NULL,
    Evalp_Min_Mark_ToPass int NULL,
    Evap_FullMark int NULL,
    Source_sys_code tinyint NULL,
    Last_Modified_Date date NULL DEFAULT (getdate()) )

-----Date Dimension
CREATE TABLE DimDate(
    DateSK int NOT NULL Primary Key,
    Date date NOT NULL,
    Day char(2) NOT NULL,
    DaySuffix varchar(4) NOT NULL,
    DayOfWeek varchar(9) NOT NULL,
    DOWInMonth tinyint NOT NULL,
    DayOfYear int NOT NULL,
    WeekOfYear tinyint NOT NULL,
    WeekOfMonth tinyint NOT NULL,
    Month char(2) NOT NULL,
    MonthName varchar(9) NOT NULL,
    Quarter tinyint NOT NULL,
    QuarterName varchar(6) NOT NULL,
    Year char(4) NOT NULL,
    StandardDate varchar(10) NULL,
    HolidayText varchar(50) NULL,
    FiscalDay char(2) NULL,
    FiscalMonth char(2) NULL,
    FiscalMonthName varchar(9) NULL,
    FiscalQuarter tinyint NULL,
    FiscalQuarterName varchar(6) NULL,
    FiscalYear char(4) NULL,
)
```

```

-----Exam Dimension
CREATE TABLE Exam_DIM(
    Exam_id int IDENTITY(1,1) NOT NULL Primary Key,
    Exam_BK int NOT NULL,
    Question_BK int NOT NULL,
    Duration int NULL,
    No_Questions int NULL,
    Date date NULL DEFAULT (getdate()),
    Min_Mark_To_Pass int NULL,
    Full_Mark int NULL,
    Question_Type varchar(50) NULL,
    Questions varchar(1000) NULL,
    Correct_Choice varchar(500) NULL,
    ST_Answers varchar(500) NULL,
    St_Degree int NULL,
    Source_sys_code tinyint NULL,
    Last_Modified_Date date NULL DEFAULT (getdate()),
)

-----Evaluation Project Fact
CREATE TABLE Evaluation_Project_Fact(
    [Evaluation_Fact_ID] [int] IDENTITY(1,1) NOT NULL Primary Key,
    [Student_FK] [int] NOT NULL,
    [Branch_FK] [int] NOT NULL,
    [Course_FK] [int] NOT NULL,
    [Project_Score] [int] NULL,
    [Source_Sys_code] [tinyint] NULL,
    [Created_AT] [date] NULL DEFAULT (getdate()),
    FOREIGN KEY(Branch_FK)REFERENCES Branches_DIM (Branch_ID),
    FOREIGN KEY(Course_FK)REFERENCES Course_Dim (Course_ID),
    FOREIGN KEY(Student_FK)REFERENCES Student_DIM (Student_ID)
)

-----Graduation Project Team Dimension
CREATE TABLE Graduation_Project_Team_DIM(
    Team_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Team_BK int NOT NULL,
    Project_BK int NOT NULL,
    Project varchar(100) NULL,
    Grade varchar(100) NULL,
    Graduation_Project_SDate date NULL,
    Graduation_Project_EDate date NULL,
    Min_Marks_ToPass int NULL,
    FullMark int NULL,
    Source_sys_code tinyint NULL,
    Last_modified_date date NULL DEFAULT (getdate())
)

-----Instructor Dimension
CREATE TABLE Instructor_DIM(
    Instructor_ID int IDENTITY(1,1) NOT NULL Primary Key,
    Instructor_BK int NOT NULL,
    first_name varchar(500) NULL,
    Last_name varchar(500) NULL,
    I_City varchar(50) NULL,
    Salary int NULL,
    Job_Type varchar(100) NULL,
    source_sys_code tinyint NULL,
    start_date datetime NULL,
    end_date datetime NULL,
    Last_Modified_date date NULL DEFAULT (getdate()),
    is_current int NULL,
)

```


-----Student Dimension

```
CREATE TABLE Student_DIM(  
    Student_ID int IDENTITY(1,1) NOT NULL Primary Key,  
    Student_BK int NOT NULL,  
    Certificate_BK int NULL,  
    Job_BK int NULL,  
    Freelance_BK int NULL,  
    Team_BK int NULL,  
    First_name varchar(100) NULL,  
    Last_Name varchar(100) NULL,  
    Graduation_Year int NULL,  
    Gender varchar(1) NULL,  
    DateOfBirth date NULL,  
    S_City varchar(50) NULL,  
    Certificate_Name varchar(50) NULL,  
    Certificate_Hours int NULL,  
    Job_Title varchar(50) NULL,  
    Hiring_Date date NULL,  
    Company_Name varchar(100) NULL,  
    Salary int NULL,  
    Job_Type varchar(50) NULL,  
    Freelance_Cost int NULL,  
    Freelance_Duration int NULL,  
    Clint_Name varchar(200) NULL,  
    Source_sys_code tinyint NULL,  
    last_date_modified date NULL DEFAULT (getdate())  
)
```

-----Examination Fact

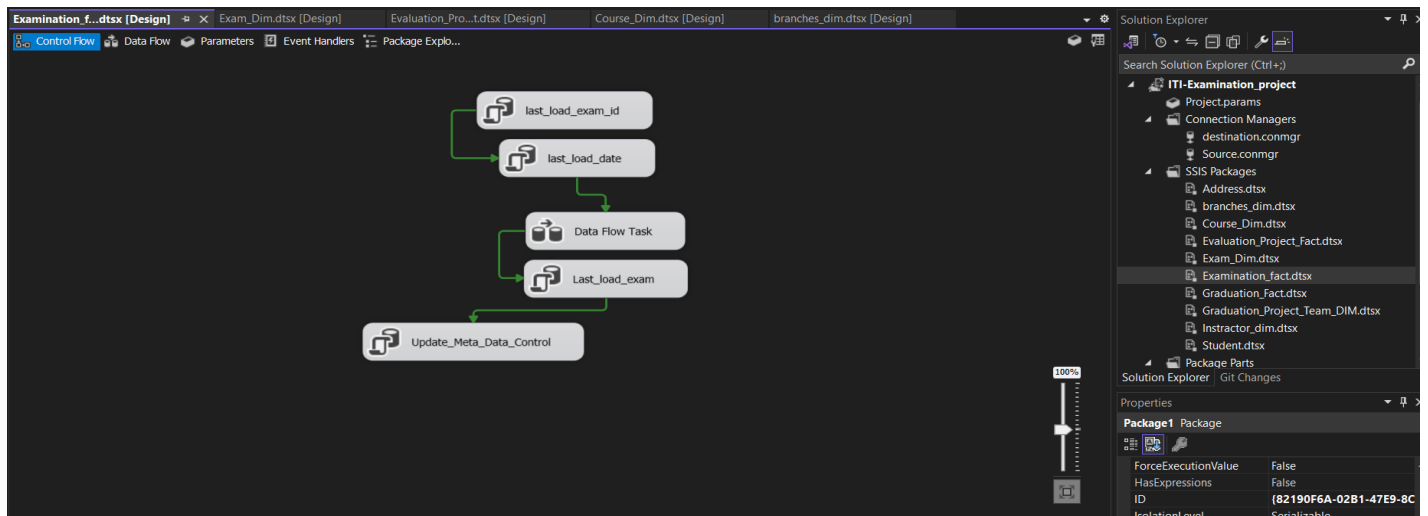
```
CREATE TABLE Examination_Fact(  
    Examination_Fact_ID int IDENTITY(1,1) NOT NULL Primary Key,  
    Student_FK int NOT NULL,  
    Course_FK int NOT NULL,  
    Instructor_FK int NOT NULL,  
    Branch_FK int NOT NULL,  
    Exam_FK int NOT NULL,  
    Date_FK int NOT NULL,  
    Exam_Degree int NULL,  
    Source_sys_code int NULL,  
    Created_AT date NULL DEFAULT (getdate()),  
    FOREIGN KEY(Branch_FK)REFERENCES Branches_DIM (Branch_ID),  
    FOREIGN KEY(Course_FK)REFERENCES Course_Dim (Course_ID),  
    FOREIGN KEY(Date_FK)REFERENCES DimDate (DateSK),  
    FOREIGN KEY(Exam_FK)REFERENCES Exam_DIM (Exam_id),  
    FOREIGN KEY(Instructor_FK)REFERENCES Instructor_DIM (Instructor_ID),  
    FOREIGN KEY(Student_FK)REFERENCES [Student_DIM] (Student_ID)  
)
```

-----Graduation Fact

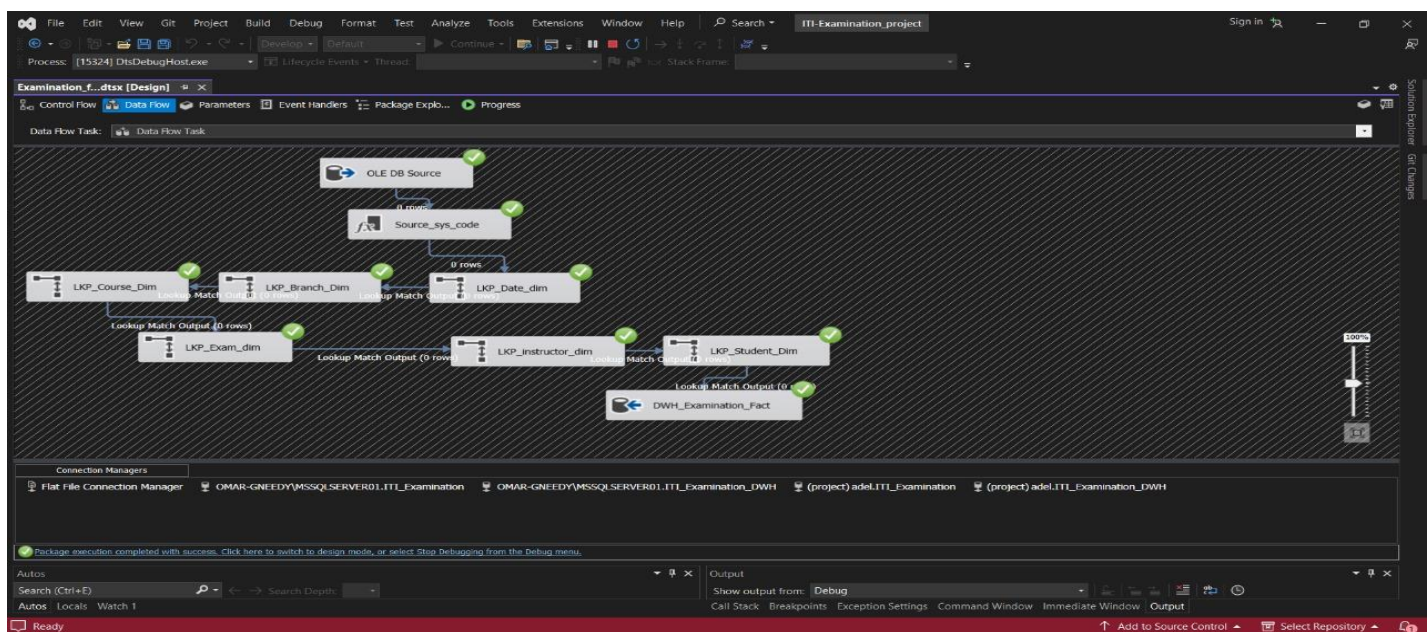
```
CREATE TABLE Graduation_Fact(  
    Graduation_Fact_ID int IDENTITY(1,1) NOT NULL Primary Key,  
    Team_FK int NOT NULL,  
    Branch_FK int NOT NULL,  
    Graduation_Project_Score int NULL,  
    Source_sys_code tinyint NULL,  
    Created_AT date NULL DEFAULT (getdate()),  
    FOREIGN KEY(Branch_FK)REFERENCES Branches_DIM (Branch_ID),  
    FOREIGN KEY(Team_FK)REFERENCES Graduation_Project_Team_DIM (Team_ID)  
)
```

5.3 “ETL Processes: Extraction, Transformation, and Loading”

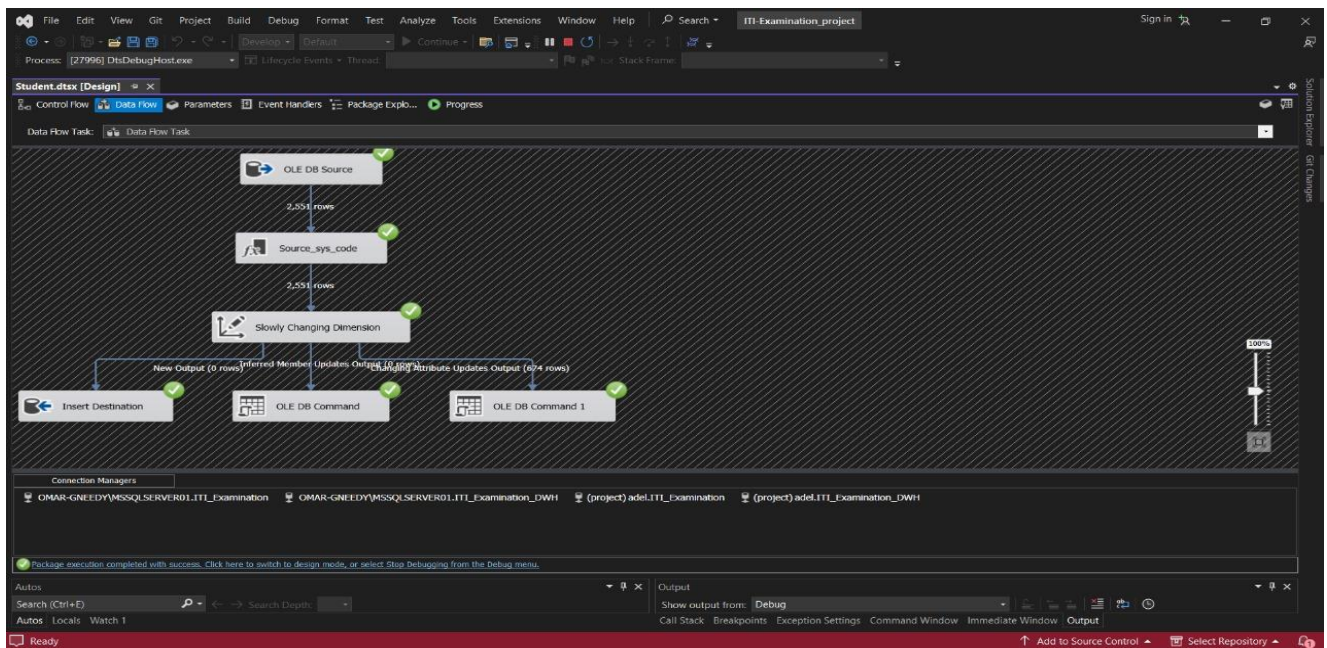
Control Flow of examination Fact:



Data Flow of examination Fact:



Data Flow of Student Dimension:



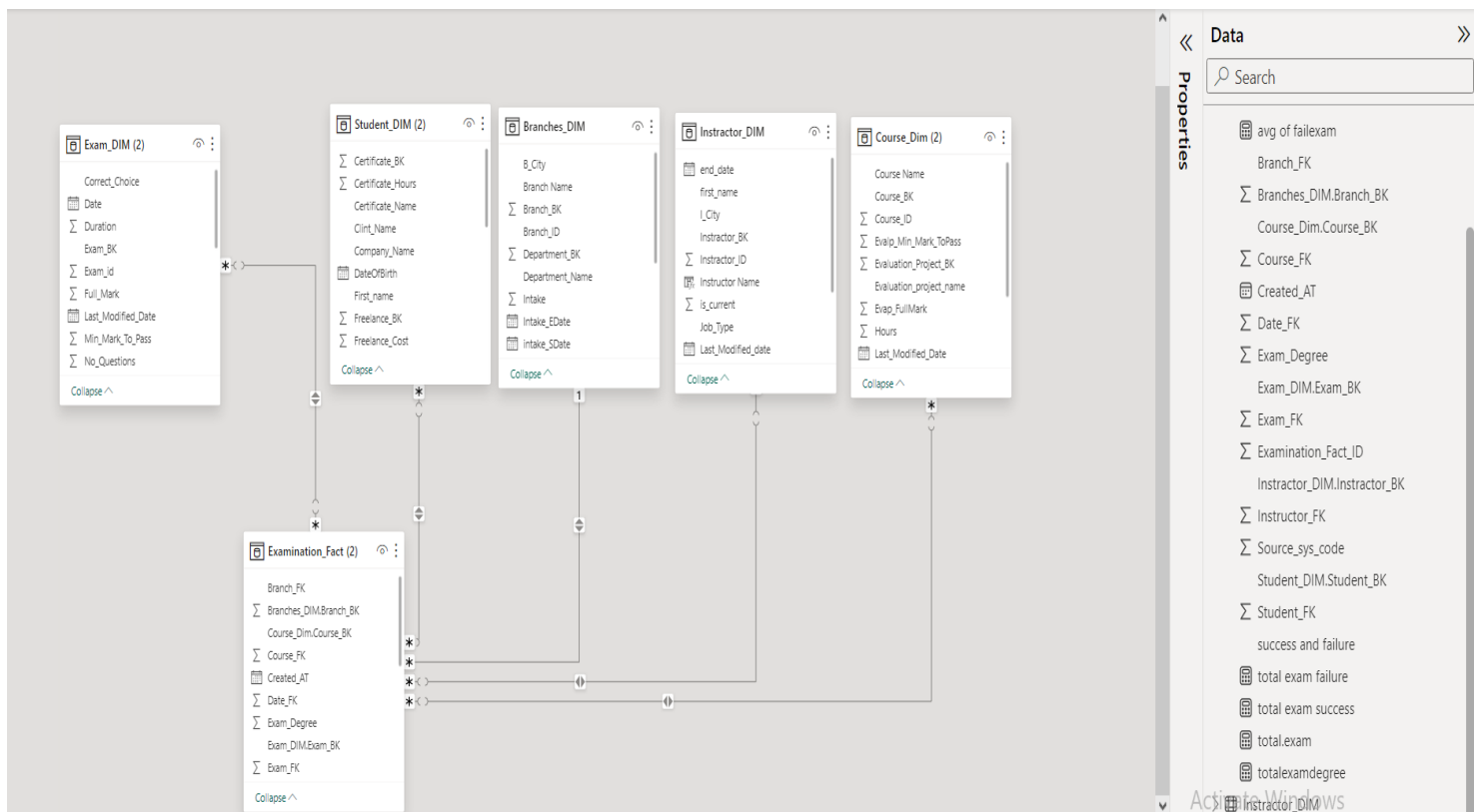
6. Power BI Integration

6.1 “Data Transformation Modelling for Analytics”

Table: AddColumn("#Expanded Instructor_DIM", "success and failure", each if [Exam_Degree] >= 10 then "pass" else "fail")

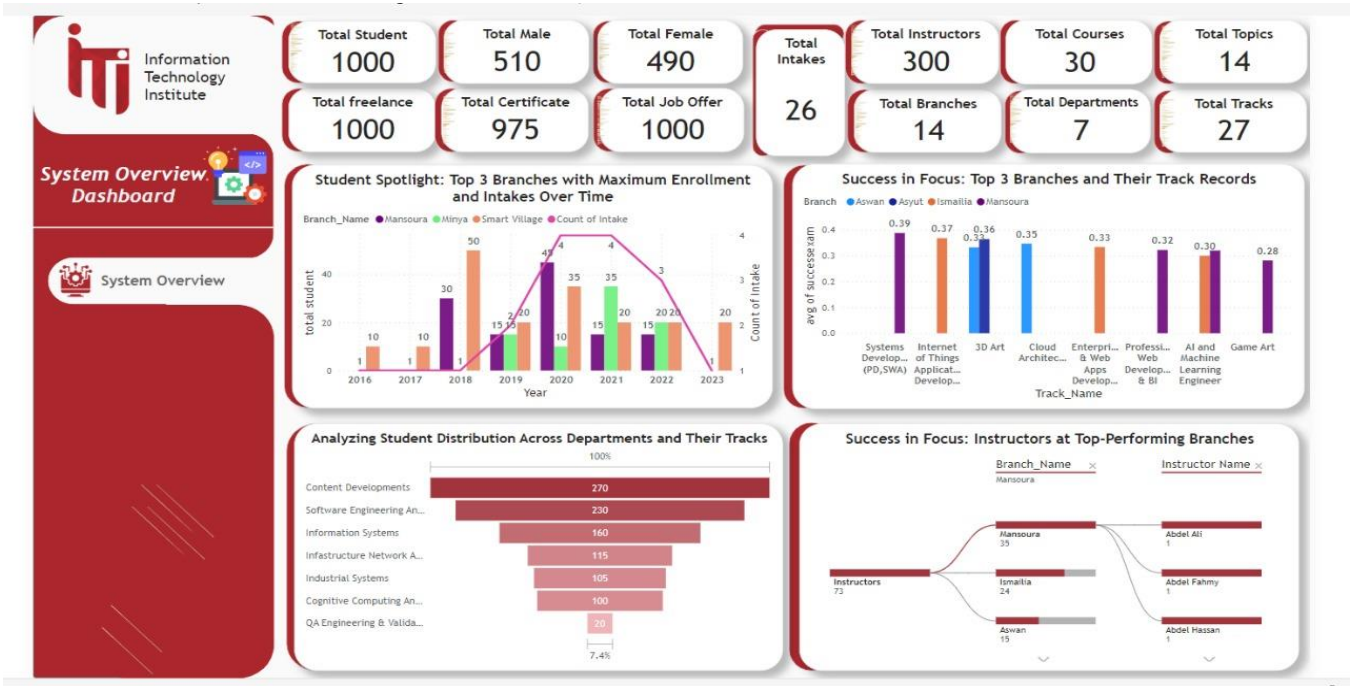
course_dim.Course_BK	DimDate	Exam_DIM.Exam_BK	Instructor_DIM.Instructor_BK	Student_DIM.Student_BK	success and failure
Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%	Valid 100% Error 0% Empty 0%
1000 distinct, 1000 unique		1000 distinct, 1000 unique	39 distinct, 0 unique	200 distinct, 0 unique	
1	10	Value	2	154	1 fail
2	11	Value	3	39	1 fail
3	12	Value	4	11	1 pass
4	13	Value	5	200	1 fail
5	24	Value	6	197	1 fail
6	10	Value	7	154	2 fail
7	11	Value	8	39	2 fail
8	12	Value	9	11	2 fail
9	13	Value	10	200	2 fail
10	24	Value	11	197	2 fail
11	10	Value	12	154	3 fail
12	11	Value	13	39	3 fail

PROPERTIES
Name: Examination_Fact (2)
All Properties
APPLIED STEPS: Source, Navigation, Expanded Exam_DIM, Expanded Course_Dim, Expanded Student_DIM, Expanded Branches_DIM, Expanded Instructor_DIM, Added Conditional Column

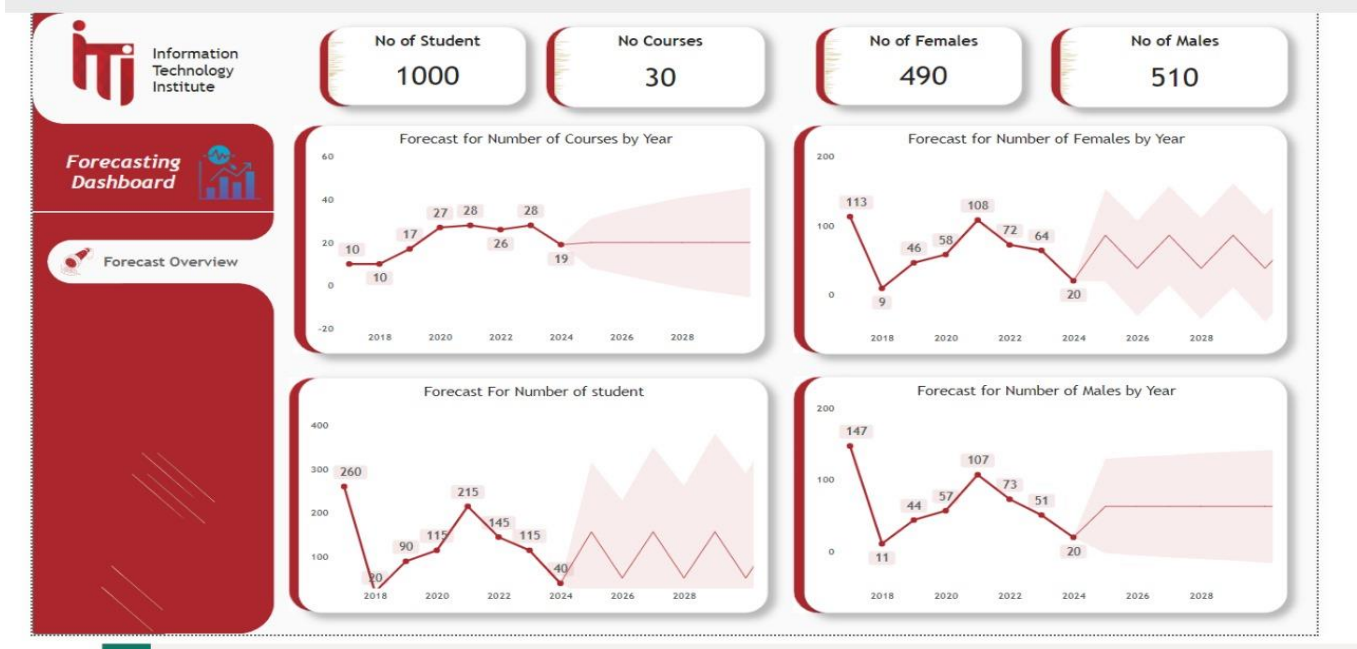


6.2 “Visualization Techniques for Data Insights”

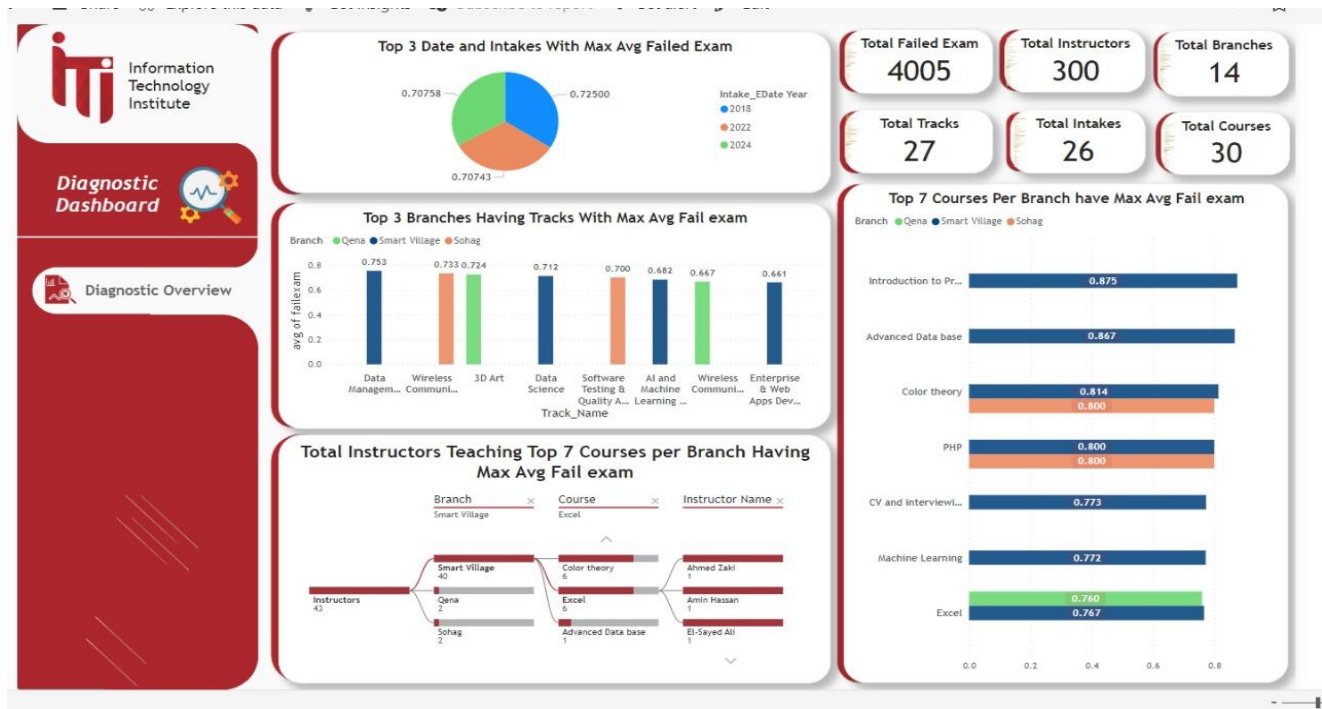
System Overview Dashboard: This Dashboard aids educators and administrators in evaluating the system's efficacy, identifying areas for improvement, and making informed decisions to enhance learning and assessment experiences.



Forecasting Dashboard: provides a dynamic platform for predicting future trends and outcomes based on historical data and current variables.



Diagnostic Dashboard: to diagnose problems, optimize workflows, prioritize tasks, and drive continuous improvement initiatives across the organization.



7. Python Application Development

7.1“Establishing Database Connections with Python”

```
1 import pyodbc
2 import pandas as pd
3 import streamlit as st
4
5
6 username = "omarauth"
7 password = "
8 server = "OMAR-GNEEDY\MSSQLSERVER01"
9 database = "ITI_Examination"
10
11 cnxn = pyodbc.connect('DRIVER={SQL Server};SERVER='+server+';DATABASE='+database+';UID='+username+';PWD='+ password)
```

7.2“Building Frontend and Backend Applications”

Backend:

the main page: leads to student or instructor logins.

```
167 def main_page():
168
169     with st.container():
170         st.header("Welcome to ITI Examination System!!")
171         st.divider()
172         st.subheader("Please choose how you want to login!!")
173
174     st.divider()
175     with st.container():
176         col1, col2, col3, col4, col5, col6 = st.columns(6)
177
178         with col3:
179             btn1 = st.button("Student")
180
181         with col4:
182             btn2 = st.button("Instructor")
183
184     if btn1:
185         st.session_state.runpage = student_page
186         st.session_state.runpage()
187         st.experimental_rerun()
188
189     if btn2:
190         st.session_state.runpage = inst_page
191         st.session_state.runpage()
192         st.experimental_rerun()
193 if 'runpage' not in st.session_state:
194     st.session_state.runpage = main_page
195 st.session_state.runpage()
```

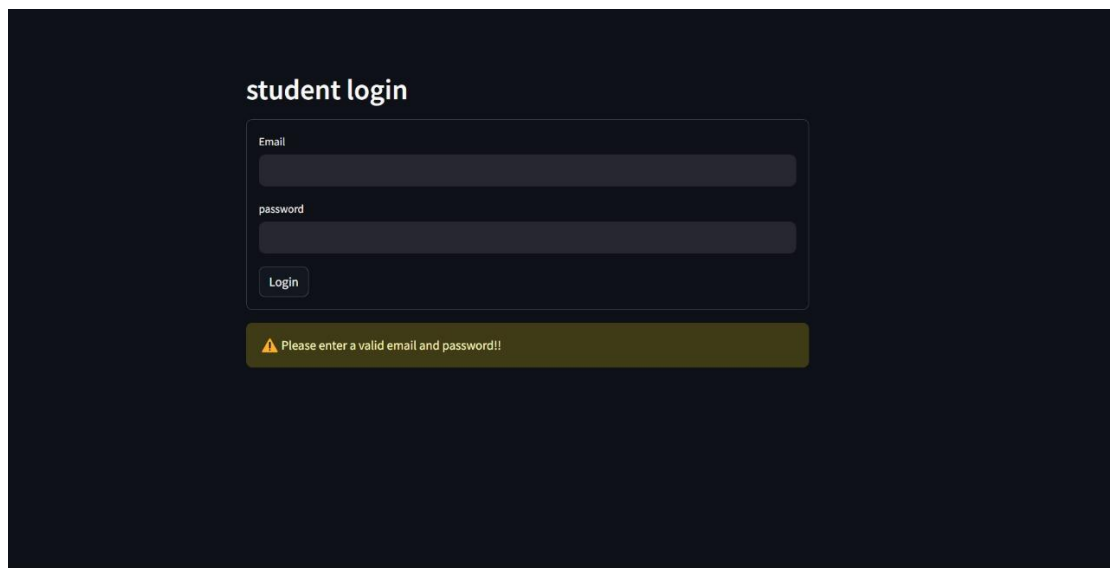
Frontend: main page



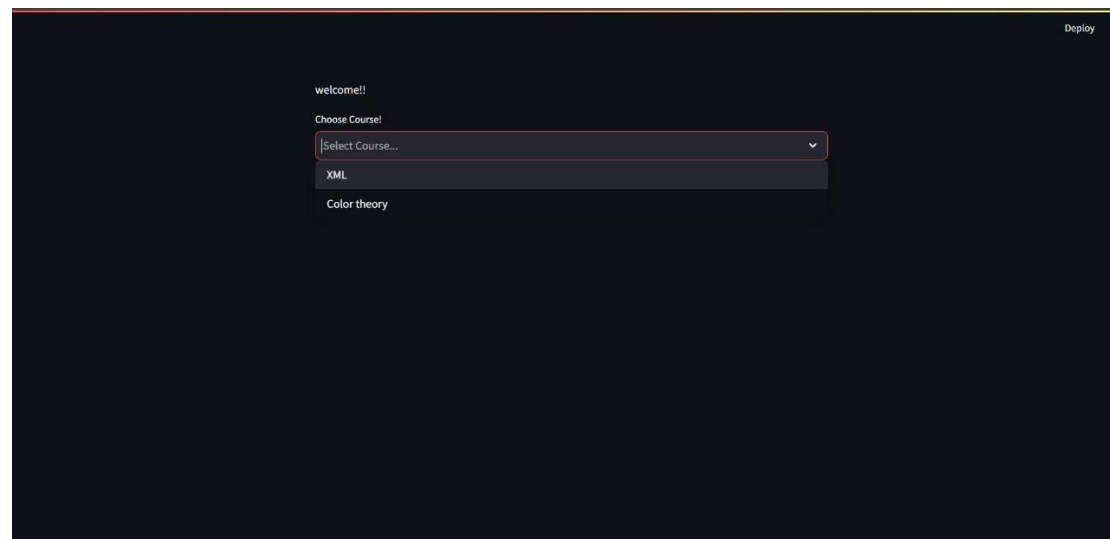
BACKEND: Student login page

```
22
23
24 def student_page():
25     st.header("student login")
26     with st.form(key="Login"):
27         email = st.text_input("Email")
28         password = st.text_input("password")
29         sub_btn = st.form_submit_button("Login")
30         query = "ValidateStudent;"
31         df_ins = pd.read_sql(query, cnxn)
32
33         for index, row in df_ins.iterrows():
34             if row['St_Email'] == email and row['Password'] == password:
35                 #st.write("success")
36                 global st_row
37                 st_row = row
38                 st.session_state.runpage = stu_cs
39                 st.session_state.runpage()
40                 st.experimental_rerun()
41             break
42         else:
43             if email != '' or password != '':
44                 st.error('Login Failed!!', icon="🚫")
45             else:
46                 st.warning('Please enter a valid email and password!!', icon="⚠️")
47
```

FRONTEND: STUDENT LOGIN



The screenshot displays a web application titled "student login". It features a dark-themed interface. A form is centered on the page, containing two input fields: "Email" and "password". Below these fields is a "Login" button. A yellow warning message with a triangle icon is displayed below the form, stating "Please enter a valid email and password!!".



The screenshot displays a web application titled "welcome!!". It features a dark-themed interface. A "Choose Course!" section is visible, containing a dropdown menu with the text "Select Course...". Below the dropdown, a list of courses is shown: "XML" and "Color theory". A "Deploy" button is located in the top right corner.

Deploy

- ☐ To define a processing instruction
- ☐ To include character data that should not be parsed

8- What is the purpose of XSLT in XML?

- ☐ To define the structure of XML documents
- ☐ To transform XML documents into other formats
- ☒ To query XML documents
- ☐ To validate XML documents against a schema

9- What is the XML declaration used for?

- ☐ To define the root element of an XML document
- ☒ To specify the character encoding of the document
- ☐ To declare namespaces used in the document
- ☐ To indicate the version of XML being used

10- What does XML stand for?

- ☐ Extensible Markup Language
- ☒ Extraordinary Markup Language
- ☐ Extended Markup Language
- ☐ Exceptional Markup Language

Submit!

Dep

Submitted!!

Your degree is 12 out of 20

Summary:

An examination system for ITI student instructors is crucial for effective tracking, programming, and departmental management.

It encompasses various components including student database management, exam scheduling, course material management, tracking of student progress, examination platform, grading and evaluation, departmental collaboration, scalability, and flexibility. By incorporating these features, institutions can efficiently manage exams, track student progress, and maintain the quality of training programs offered.

By implementing this comprehensive examination system, ITI institutions can streamline their operations, enhance the learning experience for student instructors, and ensure the delivery of high-quality training programs that meet industry standards.